

DESENLER

Desen Nedir?

Nesne yönelimli programlamanın ortaya çıkışıyla beraber, daha önceden yazılmış hazır **bileşenlerin** (**component**) **yeniden kullanımı** (**reusing**) konusunda oldukça önemli ilerlemeler sağlanmıştır. Bunun paralelinde **tecrübelerin** (**experience**) yeniden kullanımı konusunda da çeşitli çalışmalar yapılmış ve **desen** (**pattern**) kavramı ortaya çıkmıştır.

Yazılım geliştirme öncesi, geliştirme aşması ve sonrasında daha önce yaşanmış çeşitli problemlere karşı birçok kimse tarafından çeşitli çözümler getirilmiştir. Desenler, daha önce geliştirme yapan programcıların yanlış yaparak doğru yapmayı öğrendikleri başarılı tecrübelerin anlatılması için bir araçtır. Gerçekleştirilen bu çözümlerin, daha sonradan yaşanacak benzer nitelikli problemlerde kullanılması amacıyla desenler kullanılır. Desen kavramının temelleri Mimar Christopher Alexander'ın 1970 sonlarında başlattığı çalışmalara dayanmaktadır³².

1987 yılında uluslararası “Nesne Yönelimli Programlama, Sistemler, Diller ve Uygulamalar” konferansına kadar desenlerle ilgili bir çalışma ortaya çıkmamıştır³³. Bu tarihten sonra ise başta *Grady Booch*, *Richard Helm*, *Erich Gamma* ve *Kent Beck* olmak üzere pek çok bilim adamı desenlerle ilgili makale ve sunumlar yayınlamışlardır. 1994 yılında *Erich Gamma*, *Richard Helm*, *Ralph Johnson* ve *John Vlissides* tarafından yayınlanan “Tasarım Desenleri: Tekrar kullanılabilir Nesne Yönelimli Yazılımın Temelleri” kitabı, tasarım desenlerin yazılımda kullanılmasında dönüm noktası olmuştur³⁴.

Desenlerin aksine anti-desen (**anti-pattern**) ise başarısızlık tecrübelerini anlatır

Yazılımlarda kullanılan desenler yedi ana başlıkta toplanırlar. Bunlar;

1. Mimari desenler (**architectural pattern**)
2. Analiz desenleri (**analysis pattern**)
3. Tasarım desenleri (**design pattern**)
4. Kodlama desenleri (**implementation pattern**)
5. Test desenleri (**test pattern**)
6. Çözüm desenleri (**solution pattern**)
7. Veri desenleri (**data pattern**)

Bu desenlerin en önemlisi ve ilk gelişenlerden birisi, **tasarım desenleridir** (**design pattern**). Tasarım desenleri aslında, Nesne Yönelimli Tasarım Prensiplerini, yazılımlara uygulamanın bir başka ifadesidir. Bu nedenle çok önemlidir. Tasarım desenleri ise üç ana başlıkta toplanmaktadır.

1. Nesne imali ile ilgili desenler (**creational patterns**)
2. Yapısal desenler (**structural patterns**)
3. Davranışlarla ilgili desenler (**behavioral patterns**)

Desenlerin çok sık kullanıldığı programlama yöntemine günümüzde desen yönelimli programlama (**pattern oriented programming**) adı verilmektedir³⁵. İzleyen sayfalarda verilen desen UML diyagramlarının birçoğu Erich Gamma'nın Design Pattern CD yayınından alınmıştır³⁶.

Bu bölümde verilen örnekleri iyi anlamak için *Sınıflar Arası İlişkiler* başlığını iyi özümsemek gerekir. Bu bölümde oluşturulan projeler eski tip proje oluşturularak yapılmıştır. Bunun için: *Visual Studio* açılır ve şu adımlar takip edilir;

1. Programı ilk defa çalıştırtıyorsanız, karşılama sayfasından *Create a new project* seçiniz. Eğer program açık ise *File* menüsüne giderek *File → New Project* seçiniz.
2. Proje tipi olarak *Console Application* seçilir.
3. *Project Name*, *Location* ve *Solution Name* belirlenir

³² <http://gee.cs.oswego.edu/dl/ca/ca/ca.html>

³³ OOPSLA, Object Oriented Programming, Systems, Languages, and Applications

³⁴ Design Patterns: Elements of Reusable Object-Oriented Software, ISBN 0-201-63361-2

³⁵ www.mathcs.sjsu.edu/faculty/pearce/patterns.html

³⁶ Design Pattern CD, Addison-Wesley, 1998

4. *Additional Information* sayfasında *Do not use top-level statemets* kutucuğu SEÇİLİ bırakılır. Böylece geleneksel proje şablonu ile proje oluşacaktır.
5. *Solution Explorer* üzerinde *Program.cs* dosyası açılır.

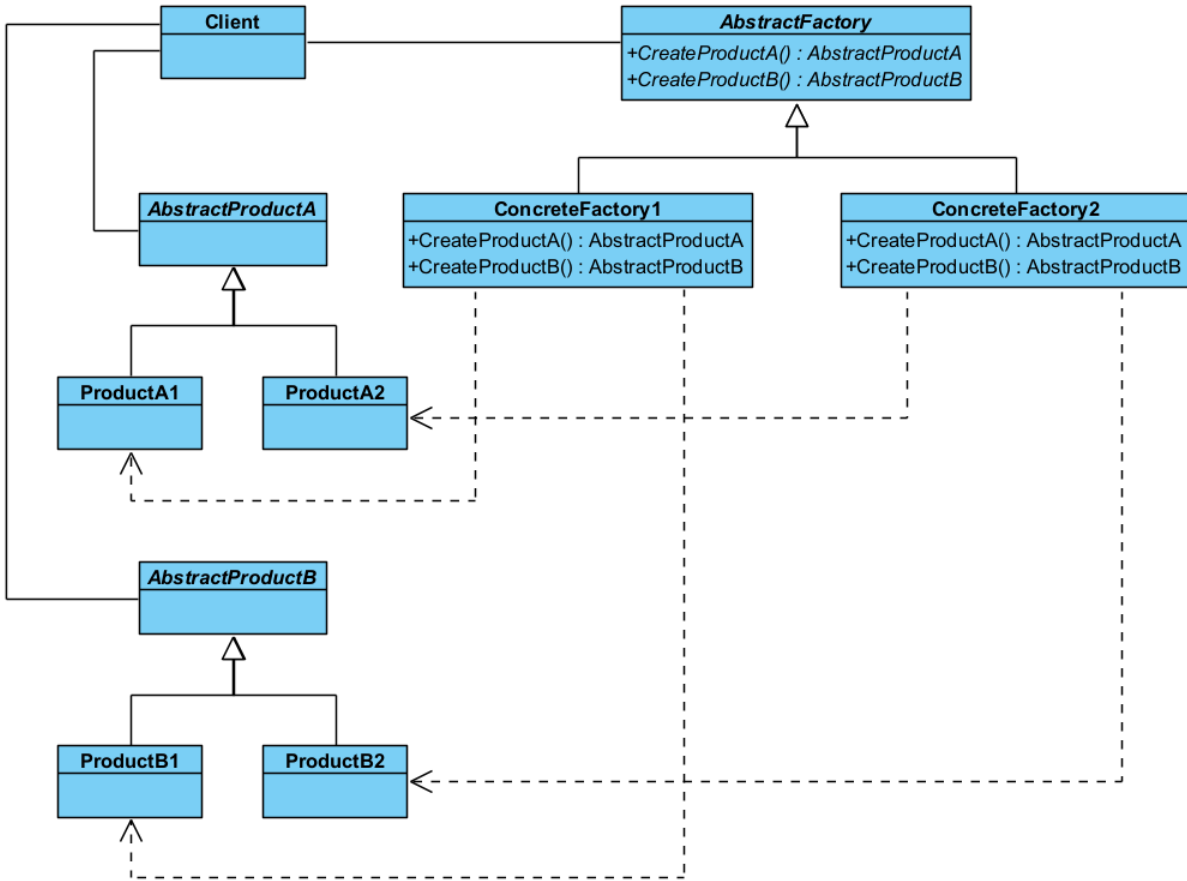
Nesne İmalatı ile İlgili Tasarım Desenleri

Nesne imalatı ile ilgili desenler (**creational patterns**), nesne (object) örnekleme ile ilgili ortaya çıkabilecek çeşitli problemlere çözüm getirmektedirler. Bu desenlerde nesneler dinamik olarak imal edilirler, çünkü değişim yönetiminde statik nesneler kullanılmazlar.

Soyut Fabrika Deseni

Soyut fabrika deseninde (**abstract factory pattern**) amaç, nesnelerin imal edilmesi ve kullanılması ile ilgili olan kısımlarının birbirinden ayrılmasıdır. Bu amaca, birbiriyle ilgili ya da birbirine bağımlı olabilecek nesnelerin imal edilmesinde, nesnelerin somut (concrete) sınıflarını değil soyut (abstract) sınıfları kullanarak ulaşılır. Somut sınıflar istemciden izole edilir ve programcıya somut sınıfları daha kolay değiştirme için olanak sağlar.

Bu başlıktaki desenlerde, nesnelerin (objects) yapılandırılıp imal edildiği (creation) yer için fabrika (factory) kavramı, üretilen nesneler için ise ürün (product) kavramı kullanılmaktadır. Aşağıdaki UML diyagramında görüldüğü üzere istemci yalnızca soyut sınıflara bağımlı olduğundan somut ürüne olan bağımlılık azaltılmıştır (loosely coupling).



Şekil 35. Soyut Fabrika Deseni UML Diyagramı

Şimdi bu deseni kodlamaya başlayalım. İlk önce **AbstractProductA** ve alt sınıflarını kodlayalım.

```

abstract class AbstractProductA
{
}
class ProductA1 : AbstractProductA
{
}
class ProductA2 : AbstractProductA

```

```
{  
}
```

Ardından **AbstractProductB** ve alt sınıflarını kodlayalım. Bu sınıftan imal edilecek ürünler **AbstractProductA** ürünleriyle etkileşim yaptığı varsayılarak bir **interact** yöntemi eklenmiştir.

```
abstract class AbstractProductB {  
    public abstract void Interact(AbstractProductA a);  
}  
class ProductB1 : AbstractProductB {  
    public override void Interact(AbstractProductA a)  
    {  
        Console.WriteLine("{0} ile {1} etkileşiyor.",  
            this.GetType().Name, a.GetType().Name);  
    }  
}  
class ProductB2 : AbstractProductB  
{  
    public override void Interact(AbstractProductA a)  
    {  
        Console.WriteLine("{0} ile {1} etkileşiyor.",  
            this.GetType().Name, a.GetType().Name);  
    }  
}
```

Ardından ürünleri imal edecek fabrika sanal sınıfı ve somut sınıflarını tanımlayalım;

```
abstract class AbstractFactory  
{  
    public abstract AbstractProductA CreateProductA();  
    public abstract AbstractProductB CreateProductB();  
}  
class ConcreteFactory1 : AbstractFactory  
{  
    public override AbstractProductA CreateProductA()  
    {  
        return new ProductA1();  
    }  
    public override AbstractProductB CreateProductB()  
    {  
        return new ProductB1();  
    }  
}  
class ConcreteFactory2 : AbstractFactory  
{  
    public override AbstractProductA CreateProductA()  
    {  
        return new ProductA2();  
    }  
    public override AbstractProductB CreateProductB()  
    {  
        return new ProductB2();  
    }  
}
```

Son olarak fabrika ve ürün sınıflarını sanal olarak kullanan istemci programı kodlayalım;

```
class Client  
{  
    static void Main(string[] args)  
    {  
        AbstractFactory factory1 = new ConcreteFactory1();  
        AbstractFactory factory2 = new ConcreteFactory2();  
        // Bu istemcide ürünlerin yaratılmasına
```

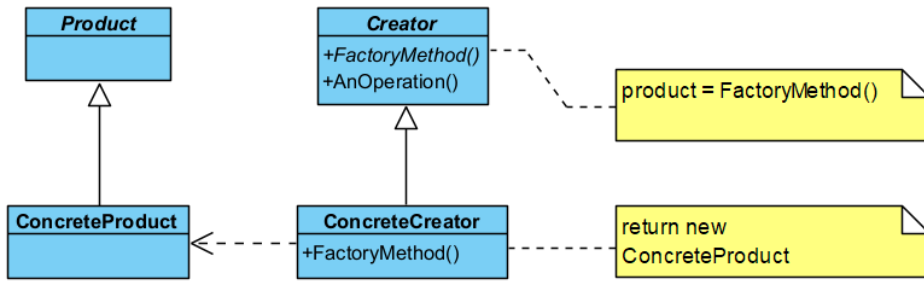
```
// ilişkin bir işlem yapılmamaktadır.
// Ürünler birbirlerine bağlı (depended) yada
// ilişkilidir (relation).
AbstractProductA producta = factory1.CreateProductA();
AbstractProductB productb = factory2.CreateProductB();
productb.Interact(producta);
}
}
/* Program Çalıştığında;
ProductB2 ile ProductA1 etkileşiyor.

...Program finished with exit code 0
*/
```

Fabrika Yöntemi Deseni

Fabrika yöntemi deseninde (factory method pattern) amaç, imal edilecek nesnenin sınıfını kesin olarak belirtmeden nesne yaratma işleminin gerçekleştirilmesidir. Bunu yapmak için fabrika yöntemi adında soyut (abstract) bir yöntem tanımlanır, fakat nesneleri imal etme (instantiation) işlemi alt sınıflara bırakılır. Yani sanal bir yapıcı (virtual constructor) kullanılır.

Soyut fabrika deseninde olduğu gibi dinamik nesne imal etme ile ilgili new anahtar kelimesi, fabrika yöntemi içinde kullanıldığından, istemcinin, ürünün yapıcısına (constructor) bağımlılığı ortadan kalkar. Aşağıdaki UML diyagramında görüldüğü üzere istemcinin somut ürünün yapıcısına bağımlılığı kaldırılmış ve nesne imal etme süreci kontrol altına alınmıştır.



Şekil 36. Fabrika Yöntemi Deseni UML Diyagramı

Şimdi bu deseni kodlamaya başlayalım. İlk önce **Product** ve alt sınıflarını kodlayalım.

```
abstract class Product { } // "ConcreteProduct", Somut ürün
class ConcreteProduct : Product { } // "Creator", Soyut Üretici
abstract class Creator
{
    // Ürün Üretecek Soyut Fabrika İşlevi
    // Bu desende bu işlev; Ürün yaratacak bir arayüzdür.
    // Bu arayüz istemcinin (client) ürün yaratmaya
    // (creation of objects)
    // ilişkin bağımlılığı ortadan kaldırır.
    public abstract Product FactoryMethod();
    public void AnOperation() { }
}
```

Ardından ürün imal edecek olan imalatçı **Creator** sınıfını kodluyoruz;

```
class ConcreteCreator : Creator // "ConcreteCreator", Somut Ürün Üreticisi
{
    // somut ürününü üreten Fabrika işlevi
    public override Product FactoryMethod()
    {
        return new ConcreteProduct();
    }
}
```

Son olarak ürün ve imalatçı sınıflarını sanal olarak kullanan istemci programı kodlayalım;

```

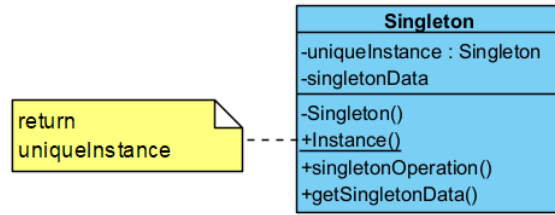
class Client
{
    static void Main(string[] args)
    {
        // Üreticileri soyut olarak kullanacağız.
        Creator creator = new ConcreteCreator();
        // Ürünleri de new kullanmadan yaratıp kullanıyoruz. Bu durumda kullanıcı ürünün
        // yaratıcı işlevine bağımlı kalmaz. Bu desenin amacı ürüne ilişkin new kelimesini
        // kullanmamaktır. Product product=new ConcreteProductA();
        // Kullanılabilir ama; Burada ürünün yapıcı işlevinin değişmesi durumunda
        // istemcinin (client) yeniden derlenmesi gerekir. İstemci ürünün nasıl yaratıldığı
        // ile değil nasıl kullanılacağı ile ilgilenir.
        Product product = creator.FactoryMethod();
        Console.WriteLine("{0} created.",product.GetType().Name);
    }
}
/* Program Çıktısı:
ConcreteProduct created.

...Program finished with exit code 0
*/

```

Tekil Nesne Deseni

Tekil Nesne deseninde (**singleton pattern**) amaç, bir sınıftan yalnızca bir örnek (**instance**) imal etmektir. Bir sınıfın yalnızca bir nesnesinin olmasına izin verilir, birden fazla olmasına izin verilmez. Yaratılan nesneye genel olarak **evrensel (global)** olarak erişilir.



Şekil 37. Tekil Deseni UML Diyagramı

Verilen UML diyagramı incelendiğinde, **Singleton** sınıfının **instance** yöntemi her seferinde aynı nesneye ilişkin referans geri döndürülür. Bu desen aynı zamanda, nesneye ilk değerlerin verilerek oluşturulduğu, imal edilme sürecini ve ilk kullanımını **izole (encapsulate)** eder. Böylece istemcilerin her biri aynı nesne ile muhatap olurlar. Şimdi **Singleton** sınıfını kodlayalım;

```

// Singleton sınıfı öyle bir sınıftır ki yalnızca bir
// örneği(instance) vardır.
class Singleton
{
    // Sınıftan yaratılan örneklerle değil,
    // sınıfa ait üye değişken (class scope member variable)
    // tanımlanıyor.
    private static Singleton uniqueInstance;
    private int singletonData;
    // Birden fazla örneğe(instance) izin vermemek için
    // yapıcı(constructor) işlevin görünürlüğü(visibility)
    // özel(private) olmalıdır.
    private Singleton() { }
    public static Singleton Instance()
    {
        if (uniqueInstance == null)
            uniqueInstance = new Singleton();
        return uniqueInstance;
    }
    public void SingletonOperation()
    {

```

```

        Console.WriteLine("Singleton İşlevi Çağrıldı.");
    }
    public int GetSingletonData()
    {
        return singletonData;
    }
}

```

Bu sınıfı kullanan istemci aşağıdaki gibi yazılabilir;

```

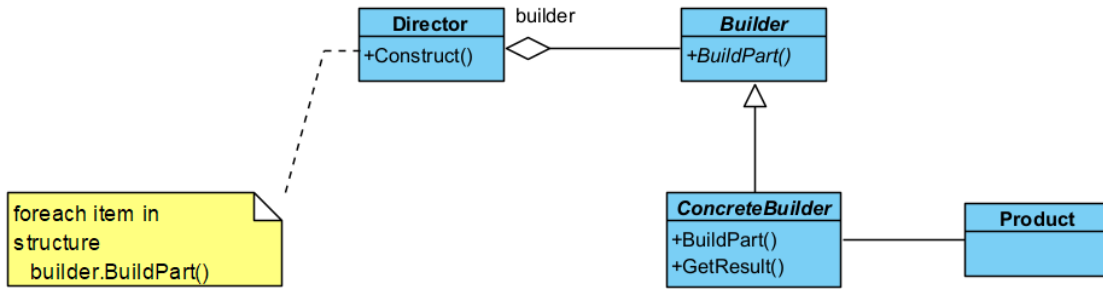
class Client
{
    static void Main(string[] args)
    {
        Singleton object1 = Singleton.Instance();
        Singleton object2 = Singleton.Instance();
        if (object1 == object2)
            Console.WriteLine
                ("object1 ve object2 AYNI nesnedir.");
        else
            Console.WriteLine
                ("object1 ve object2 FARKLI nesnedir.");
        object1.SingletonOperation();
    }
}
/* Program Çıktısı:
object1 ve object2 AYNI nesnedir.
Singleton İşlevi Çağrıldı.

...Program finished with exit code 0
*/

```

Kurucu Deseni

Kurucu deseninde (**builder pattern**) amaç, çok karmaşık bir nesnenin imal edilmesiyle ilgili işlemleri bir başka sınıfa bırakmaktır. Böylece nesnenin gösterimi ve kullanılması ilişkin kodlar ile yaratılmasına ilişkin kodlar birbirinden ayrılmış olur.



Şekil 38. Kurucu Deseni UML Diyagramı

Yukarıda verilen UML diyagramı incelendiğinde, Ürün imalatını yönetecek olan **Director** nesnesi, karmaşık nesneyi oluşturacak küçük **ürünlerin** (**product**) yapımını **Builder** nesnesine bırakır. **Director** nesnesi, oluşturulan bu küçük parçaları **Construct** yöntemi ile bir araya getirir. İlk önce Product sınıfını tanımlayalım.

```

class Product
{
    ArrayList parts = new ArrayList();
    public void Add(string part)
    {
        parts.Add(part);
    }
    public void Show()
    {
    }
}

```

```
        Console.WriteLine("Ürün Parçaları:");  
        foreach (string part in parts) Console.WriteLine(part);  
    }  
}
```

Buna ek olarak **Builder** ve alt sınıfını tanımlayalım;

```
abstract class Builder  
{  
    public abstract void BuildPartA();  
    public abstract void BuildPartB();  
    public abstract Product GetResult();  
}  
class ConcreteBuilder1 : Builder  
{  
    private Product product = new Product();  
    public override void BuildPartA()  
    {  
        product.Add("A Parçası");  
    }  
    public override void BuildPartB()  
    {  
        product.Add("B Parçası");  
    }  
    public override Product GetResult()  
    {  
        return product;  
    }  
}  
class ConcreteBuilder2 : Builder  
{  
    private Product product = new Product();  
    public override void BuildPartA()  
    {  
        product.Add("X Parçası");  
    }  
    public override void BuildPartB()  
    {  
        product.Add("Y Parçası");  
    }  
    public override Product GetResult()  
    {  
        return product;  
    }  
}
```

Şimdi de **Director** sınıfını tanımlayalım;

```
class Director  
{  
    public void Construct(Builder builder)  
    {  
        builder.BuildPartA();  
        builder.BuildPartB();  
    }  
}
```

Son olarak istemciyi tanımlayalım;

```
class Client  
{  
    static void Main(string[] args)  
    {  
        Director director = new Director();  
    }  
}
```

```

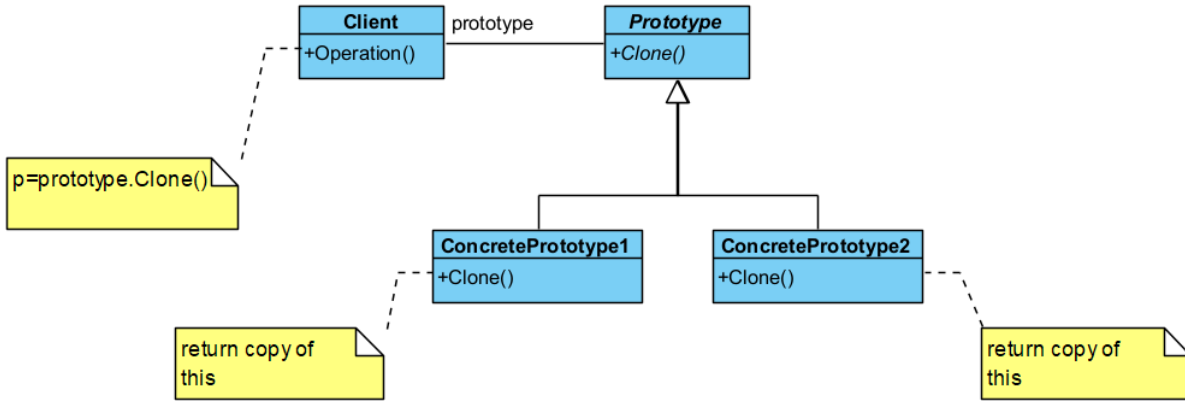
    Builder b1 = new ConcreteBuilder1();
    Builder b2 = new ConcreteBuilder2();
    director.Construct(b1);
    Product p1 = b1.GetResult();
    p1.Show();
    director.Construct(b2);
    Product p2 = b2.GetResult();
    p2.Show();
}
}
/*Programın Çıktısı:
Ürün Parçaları:
A Parçası
B Parçası
Ürün Parçaları:
X Parçası
Y Parçası

...Program finished with exit code 0
*/

```

Prototip Deseni

Prototip deseni (prototype pattern), hâlihazırda mevcut olan nesnenin bir kopyasını çıkarmak için kullanılır. Bu desen, imal edilmesi güç olan nesnelere karar verilerek, bu nesneleri tekrar imal etmek için bir sürü işlem yapılması yerine, mevcut nesneden bir kopya çıkararak kullanılmasını sağlamaktadır.



Şekil 39. Prototip Deseni UML Diyagramı

Klonlama yoluyla var olan nesnelerin bir şablonuna dayalı yeni nesneler oluşturmak için kullanılan bu desen kodlaması aşağıdaki şekilde yapılabilir;

```

abstract class Prototype
{
    private string id;
    public Prototype(string id)
    {
        this.id = id;
    }
    public string Id
    {
        get { return id; }
    }
    public abstract Prototype Clone();
}
class ConcretePrototype1 : Prototype
{
    public ConcretePrototype1(string id) : base(id) { }
    public override Prototype Clone()

```



```

    {
        return (Prototype)this.MemberwiseClone();// deep copy
    }
}
class ConcretePrototype2 : Prototype
{
    public ConcretePrototype2(string id) : base(id) { }
    public override Prototype Clone()
    {
        return (Prototype)this.MemberwiseClone();// deep copy
    }
}

```

Şimdi de istemciyi yazalım;

```

class Client
{
    static void operation() {
        ConcretePrototype1 p1 = new ConcretePrototype1("I");
        ConcretePrototype1 c1 = (ConcretePrototype1)p1.Clone();
        Console.WriteLine("{0} Klonandı.", c1.Id);
        ConcretePrototype2 p2 = new ConcretePrototype2("II");
        ConcretePrototype2 c2 = (ConcretePrototype2)p2.Clone();
        Console.WriteLine("{0} Klonlandı.", c2.Id);
    }
    static void Main(string[] args)
    {
        operation();
    }
}
/* Program Çıktısı:
I Klonandı.
II Klonlandı.

...Program finished with exit code 0
*/

```

Nesneleri kopyalama söz konusu olduğunda, genel olarak üç tür kavramdan bahsedilir;

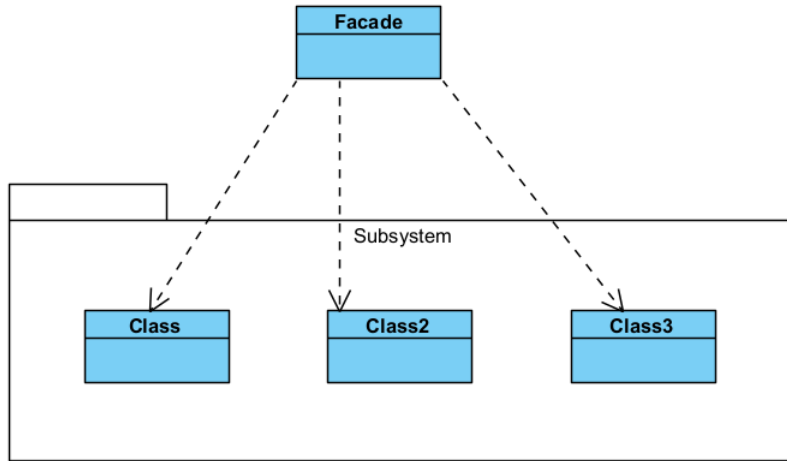
- **Üstünkörü kopyalamada** (*shallow copy*) yeni nesne eskisini referans gösterir. Bu durumda birinci nesnenin bir **alanı** (*field*) değiştiğinde ikincisinin ki de değişmiş olur. Yani birinde olan değişiklik diğeri etkiler. Bu kopyalamanın diğer adı da **bit düzeyi kopyalamadır** (*bitwise copy*).
- **Derinlemesine kopyalamada** (*deep copy*) ise birinci nesnenin aynısı bir başka bellek bölgesinde oluşturulur ve ikinci nesne yeni bellek bölgesini referans gösterir. Bu kopyalamanın diğer adı **da üye düzeyi kopyalamadır** (*memberwise copy*). Kopyalama sonrasında bir nesne üzerinde olan değişiklik yeni nesneyi etkilemez.
- Üçüncü tip bir kopyalama ise **tembel kopyalamadır** (*lazy copy*). Yukarıda bahsi geçen iki kopyalamanın iç içe girmiş şeklidir.

Yapısal Tasarım Desenleri

Yapısal desenler (*structural patterns*), sınıflar ve nesnelerin çok olduğu daha karmaşık programlarda getirilen yapısal çözüm yöntemleridir.

Vitrin Deseni

Vitrin deseninde (*facade pattern*) alt sistemlerin değişmesi halinde tüm sistemin yeniden derlenme olasılığını bertaraf etmek amacıyla alt sistemlere erişim tek bir sınıf üzerinden yapılır. Çok katmanlı yazılım mimarisinde, alt katmanların içinde olan değişiklikler sistem genelini etkilemez. Çünkü katmanlar birbirlerinin ön yüzünü (*facade*) görür ve kullanırlar.



Şekil 40. Vitrin Deseni UML Diyagramı

Bir sistem içindeki bir dizi ara yüze tek bir ara yüz üzerinden erişim sağlayacağımız program için bir dizi alt sistem kodunu yazalım;

```

class SubSystemOne // Subsystem ClassA"
{
    public void MethodOne()
    {
        Console.WriteLine(" SubSystemOne Yöntemi");
    }
}
class SubSystemTwo // Subsystem ClassB"
{
    public void MethodTwo()
    {
        Console.WriteLine(" SubSystemTwo Yöntemi");
    }
}
class SubSystemThree // Subsystem ClassC"
{
    public void MethodThree()
    {
        Console.WriteLine(" SubSystemThree Yöntemi");
    }
}
class SubSystemFour // Subsystem ClassD"
{
    public void MethodFour()
    {
        Console.WriteLine(" SubSystemFour Yöntemi");
    }
}
  
```

Şimdi de **Facade** sınıfını kodlayalım;

```

class Facade
{
    SubSystemOne one;
    SubSystemTwo two;
    SubSystemThree three;
    SubSystemFour four;
    public Facade()
    {
        one = new SubSystemOne();
        two = new SubSystemTwo();
        three = new SubSystemThree();
        four = new SubSystemFour();
    }
}
  
```

```

    }
    public void MethodA()
    {
        Console.WriteLine("MethodA() ---- ");
        one.MethodOne();
        two.MethodTwo();
        four.MethodFour();
    }
    public void MethodB()
    {
        Console.WriteLine("MethodB() ---- ");
        two.MethodTwo();
        three.MethodThree();
    }
}

```

Son olarak istemci kodumuzu yazalım;

```

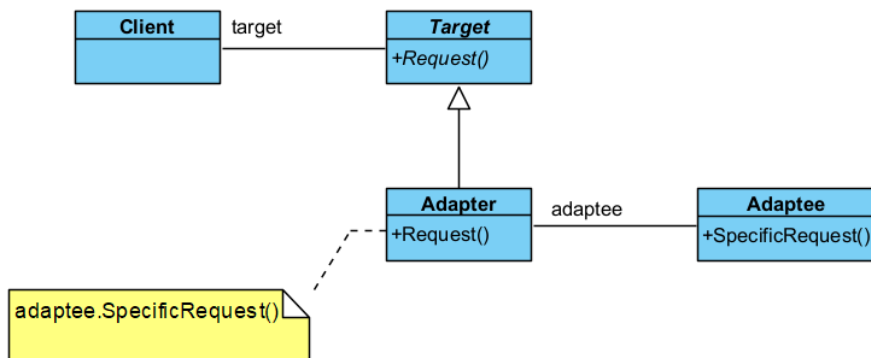
class Client
{
    static void Main(string[] args)
    {
        Facade facade = new Facade();
        facade.MethodA();
        facade.MethodB();
    }
}
/*Program çalıştığında:
MethodA() ----
SubSystemOne Yöntemi
SubSystemTwo Yöntemi
SubSystemFour Yöntemi
MethodB() ----
SubSystemTwo Yöntemi
SubSystemThree Yöntemi

...Program finished with exit code 0
*/

```

Adaptör Deseni

Adaptör deseni (**adapter pattern**), yabancı bir ara yüzü istemcinin anlayacağı bir ara yüze dönüştürür. Yani farklı ara yüzlere sahip sınıfların, iletişim ve etkileşim kurabilecekleri ortak bir nesne oluşturarak birlikte çalışmalarına izin verir. Bu desen, yabancı ara yüze yaptırılacak işin bir şekilde yapıldığı ve zaten yapılan bu işlemin istemcinin isteğine uygun hale getirildiği düşünülmelidir.



Şekil 41. Adaptör Deseni UML Diyagramı

Aşağıdaki örnekte **Target** nesnesi, istemci tarafından gelen isteği, bu işi yapabilme yeteneğine sahip **Adaptee** nesnesine yaptıracaktır. İşte burada **Adapter** nesnesi devreye girerek **Target** nesnesine gelen istekleri, **Adaptee** nesnesinin yapabileceği şekilde uyarlar.

İlk önce yabancı ara yüzü temsil eden **Adaptee** sınıfını kodluyoruz;

```
class Adaptee
{
    public void SpecificRequest()
    {
        Console.WriteLine(
            "Adaptee.SpecificRequest() işlevi çağrıldı.");
    }
}
```

Daha sonra soyut olan **Target** ve somut olan **Adapter** sınıfını kodluyoruz;

```
class Target
{
    public virtual void Request()
    {
        Console.WriteLine("Target.Request() işlevi çağrıldı.");
    }
}
class Adapter : Target
{
    private Adaptee adaptee = new Adaptee();
    public override void Request()
    {
        adaptee.SpecificRequest();
    }
}
```

Son olarak istemci tarafını kodluyoruz;

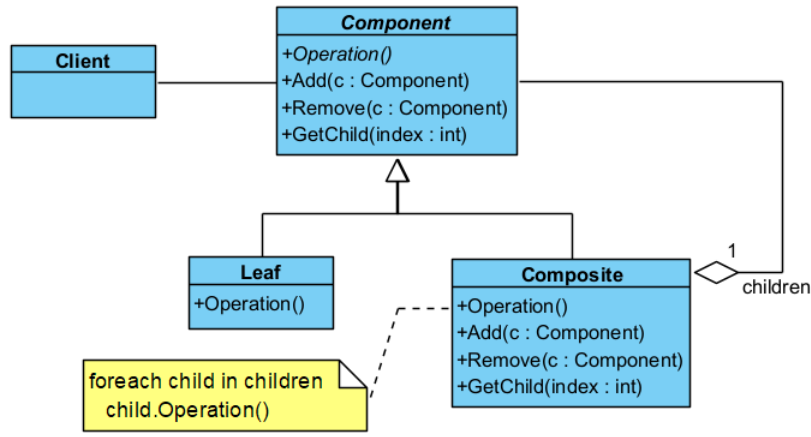
```
class Client
{
    static void Main(string[] args)
    {
        Target target = new Adapter();
        target.Request();
        // İstemci target.Request() işlevine bağımlıdır
        // Adaptöre uyan Adaptee.SpecificRequest() işlevine
        // bağımlı değildir.
        // Böylece adaptöre uyum gösteren neslelerdeki işlevler
        // istemciden bağımsız olarak değiştirilebilir.
    }
}
/* Program Çıktısı:
Adaptee.SpecificRequest() yöntemi çağrıldı.

...Program finished with exit code 0
*/
```

Bileşik Deseni

Bileşik desen (**composite pattern**), özyinelemeli (**recursive**) ya da hiyerarşik yapıya sahip nesne oluşturmak gerektiğinde yapısal olarak nasıl bir kodlama yöntemi izleneceğini söyler.

Bu desen, her nesnenin bağımsız olarak veya aynı ara yüz üzerinden iç içe geçmiş nesneler kümesi olarak ele alınabileceği nesne hiyerarşilerinin oluşturulmasını kolaylaştırır. Yukarıda verilen UML diyagramı incelendiğinde; **Leaf**, ağaç yapısı içinde kullanılabilecek, parçalara ayrılamayan en küçük bileşen olan **en ilkel** (**primitive**) yapıdır. **Component**, bu ilkel yapılardan oluşturulacak karmaşık **Composite** nesnesine ilkel yapıların eklenip çıkarılmasını sağlayacak bileşendir.



Şekil 42. Bileşik Deseni UML Diyagramı

Şimdi sanal **Component** sınıfı ile somut sınıflarını tanımlayalım;

```

abstract class Component
{
    protected string name;
    public Component(string name)
    {
        this.name = name;
    }
    public abstract void Add(Component c);
    public abstract void Remove(Component c);
    public abstract void Display(int depth);
}

class Leaf : Component
{
    public Leaf(string name) : base(name) { }
    public override void Add(Component c)
    {
        Console.WriteLine("Bileşen eklenemiyor.");
    }
    public override void Remove(Component c)
    {
        Console.WriteLine("Bileşen Silinemiyor.");
    }
    public override void Display(int depth)
    {
        Console.WriteLine(new String('-', depth) + name);
    }
}

class Composite : Component
{
    private ArrayList children = new ArrayList();
    public Composite(string name) : base(name) { }
    public override void Add(Component component)
    {
        children.Add(component);
    }
    public override void Remove(Component component)
    {
        children.Remove(component);
    }
    public override void Display(int depth)
    {
        Console.WriteLine(new String('-', depth) + name);
        foreach (Component component in children) // Recursively display child nodes
        {
            component.Display(depth + 2);
        }
    }
}
  
```

```

    }
}
}

```

Şimdi istemci kodunu yazabiliriz;

```

class Client
{
    static void Main(string[] args)
    {
        Composite root = new Composite("kök");
        root.Add(new Leaf("A Yaprığı"));
        root.Add(new Leaf("B Yaprığı"));
        Composite comp = new Composite("X Bileşiği");
        comp.Add(new Leaf("XA Yaprığı"));
        comp.Add(new Leaf("XB Yaprığı"));
        root.Add(comp);
        root.Add(new Leaf("C Yaprığı"));
        Leaf leaf = new Leaf("D Yaprığı");
        root.Add(leaf);
        root.Remove(leaf);
        // İçi içe (Recursively) ağacı göster
        root.Display(1);
    }
}
/*Program Çıktısı:
-kök
---A Yaprığı
---B Yaprığı
---X Bileşiği
----XA Yaprığı
----XB Yaprığı
---C Yaprığı

...Program finished with exit code 0
*/

```

Dekorator Deseni

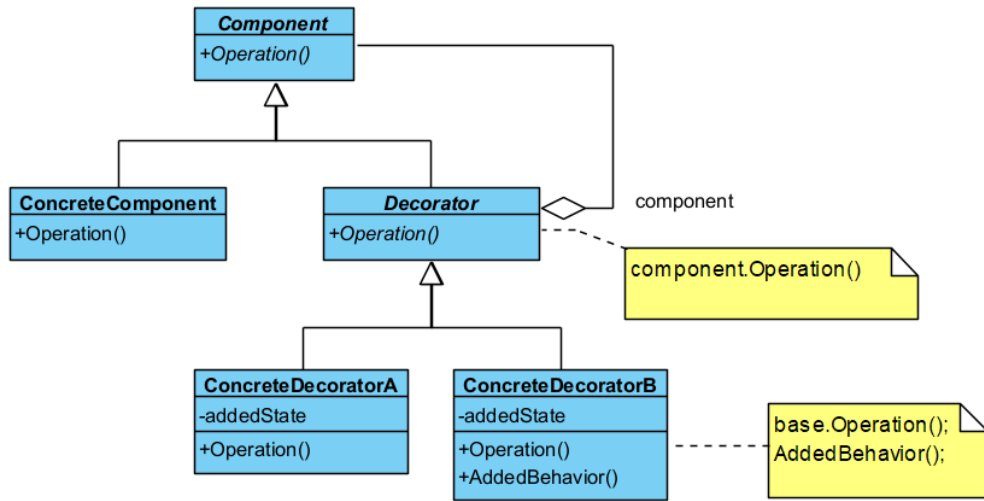
Dekorator deseninde (**decorator pattern**), bir nesneye dinamik olarak yeni durum ve davranışları eklemek mümkündür. Yani çalıştırma anında, bir nesnenin sahip olduğu yeteneklere, yeni yeteneklerin eklenmesini sağlar. Aşağıdaki UML diyagramı incelendiğinde, **Component** nesnesi, **Decorator** nesnesi aracılığı ile **çalıştırma anında** (**run time**), **addedBehavior** yöntemine ve **addedState** durumuna (**state**) sahip olur.

İlk olarak sanal sınıf olan **Component** ve somut **ConcreteComponent** bileşenlerini kodlayalım;

```

abstract class Component
{
    public abstract void Operation();
}
class ConcreteComponent : Component
{
    public override void Operation()
    {
        Console.WriteLine("ConcreteComponent.Operation()");
    }
}

```



Şekil 43. Dekorator Deseni UML Diyagramı

Şimdi de somut **Decorator** sınıfı ve alt sınıflarını kodlayalım;

```

abstract class Decorator : Component
{
    protected Component component;
    public Component SetComponent(Component component)
    {
        this.component = component;
        return this;
    }
    public override void Operation()
    {
        if (component != null)
        {
            component.Operation();
        }
    }
}

class ConcreteDecoratorA : Decorator
{
    private string addedState;
    public ConcreteDecoratorA()
    {
        addedState = "AddedState";
    }
    public string AddedState
    {
        get { return addedState; }
        set { addedState = value; }
    }
    public override void Operation()
    {
        base.Operation();
        Console.WriteLine(
            "Bileşen Yeni bir Duruma Sahiptir:{0}",
            this.AddedState);
    }
}

class ConcreteDecoratorB : Decorator
{
    void AddedBehavior()
    {
        Console.WriteLine(
            "Bileşen Yeni Bir Davranışa Sahiptir:AddedBehaviour()");
    }
}

```

```

public override void Operation()
{
    base.Operation();
    AddedBehavior();
}
}

```

Son olarak istemcimizi kodluyoruz;

```

class Client
{
    static void Main(string[] args)
    {
        Component c = new ConcreteComponent();
        ConcreteDecoratorA d1 = new ConcreteDecoratorA();
        ConcreteDecoratorB d2 = new ConcreteDecoratorB();
        c = d1.SetComponent(c); // "c" dinamik olarak yeni bir duruma sahip oldu.
        c.Operation();
        c = d2.SetComponent(c); // "c" dinamik olarak yeni bir davranışa sahip oldu.
        c.Operation();
    }
}
/* Program Çıktısı:
ConcreteComponent.Operation()
Bileşen Yeni bir Duruma Sahiptir:AddedState
ConcreteComponent.Operation()
Bileşen Yeni bir Duruma Sahiptir:AddedState
Bileşen Yeni Bir Davranışa Sahiptir:AddedBehaviour()

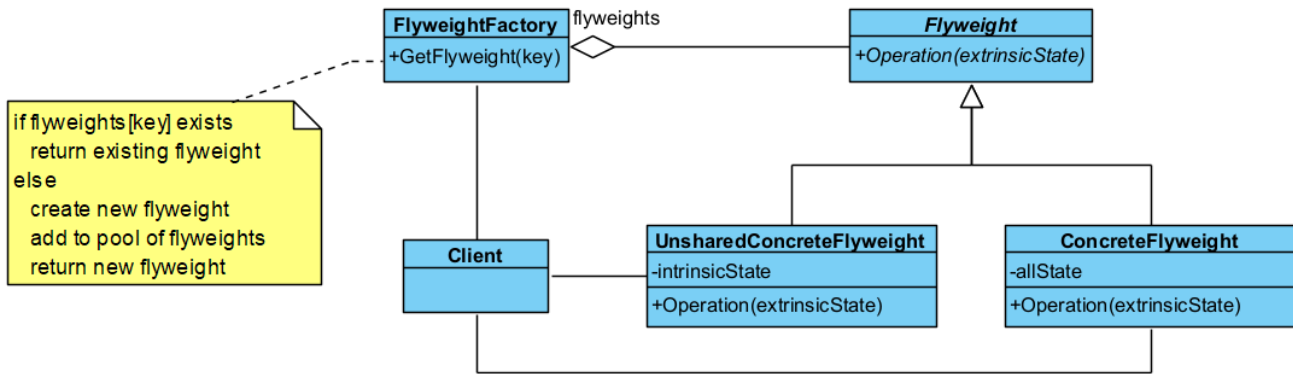
...Program finished with exit code 0
*/

```

Sineksiklet Deseni

Sineksiklet deseninde (flyweight pattern), birçok küçük nesneden oluşan bir sistemi paylaşarak kullanma hedeflenmiştir. Temel olarak sineksiklet, paylaşılan bir nesnedir ve eş zamanlı olarak küçük nesneleri kullanır. Yani sineksiklet nesnesi sözü geçen her bir küçük nesne gibi davranır.

Bu desen, bir metin editörü ile açıklanabilir. Metin editöründe yazılacak her bir harf, font bilgisine ve büyüklüğe sahiptir. İşte buradaki her bir harf ile kastedilen, sineksiklet deseninde konu olan küçük nesnelerdir. Bu nesneler bir araya gelerek satırları ve kolonları oluşturur. Sineksiklet nesnesi ise yerine göre bu harflerin her birinin yerine geçer.



Şekil 44. Sineksiklet Deseni UML Diyagramı

Yukarıdaki UML dokümanı incelendiğinde; **FlyweightFactory** küçük nesnelerden büyük resmi oluşturan fabrikayı temsil etmektedir. Birçok uygulamada durumlar geçicidir (extrinsic state) ve bu geçici durumlar saklanmazlar. Geçici durumlarla yapılan işlemler sonunda, kalıcı durumlar (intrinsic state) olur ve nesnelerde saklanır. Geçici durumlarla şekillenen ortak bir nesne (shared object) tanımıyla küçük nesneler tanımlanır. Bu küçük nesnelerin her biri sineksiklet (flyweight) olarak

adlandırılır. Ortak özellik taşımayan nesneler (unshared object) de büyük resim içinde yer alabilir. Bu nesneler de ortaklık dışı sineksiklet (unshared flyweight) olarak adlandırılır. Fabrika geçici durumlarla şekillenen sineksiklet nesneler üretir ve büyük resmi oluşturur.

Sineksiklet nesneleri ortak kullanıldığından, uygulama nesnenin kimliğine bağımlı değildir. Bu desen çok sık kullanılmaz ancak aşağıdaki durumların çoğu varsa kullanılabilir;

- Bir uygulama geniş çaplı olup çok miktarda nesne içeriyorsa,
- Nesneleri durumlarıyla olduğu gibi saklamanın maliyeti yüksekse,
- Nesnelerin birçok durumu geçici ve saklanmasına gerek yoksa,
- Birçok nesneden oluşan bir nesne grubu, ortak kullanılan bir nesnedeki geçici değişime bağlı olarak değişebiliyorsa,

İlk önce soyut **Flyweight** sınıfı ve somut sınıflarını kodlayalım;

```
abstract class Flyweight
{
    public abstract void Operation(int extrinsicstate);
}
class ConcreteFlyweight : Flyweight
{
    public override void Operation(int extrinsicstate)
    {
        Console.WriteLine("ConcreteFlyweight: " + extrinsicstate);
    }
}
class UnsharedConcreteFlyweight : Flyweight
{
    public override void Operation(int extrinsicstate)
    {
        Console.WriteLine("UnsharedConcreteFlyweight: " + extrinsicstate);
    }
}
```

Şimdi de **FlyweightFactory** sınıfını kodlayalım;

```
class FlyweightFactory
{
    private Hashtable flyweights = new Hashtable();
    public FlyweightFactory()
    {
        flyweights.Add("X", new ConcreteFlyweight());
        flyweights.Add("Y", new ConcreteFlyweight());
        flyweights.Add("Z", new ConcreteFlyweight());
    }
    public Flyweight GetFlyweight(string key)
    {
        return ((Flyweight)flyweights[key]);
    }
}
```

Son olarak de istemci sınıfı yazalım;

```
class Client
{
    static void Main(string[] args)
    {
        int extrinsicstate = 22; // Arbitrary extrinsic state
        FlyweightFactory f = new FlyweightFactory();
        // Work with different flyweight instances
        Flyweight fx = f.GetFlyweight("X");
        fx.Operation(--extrinsicstate);
        Flyweight fy = f.GetFlyweight("Y");
        fy.Operation(--extrinsicstate);
    }
}
```

```

    Flyweight fz = f.GetFlyweight("Z");
    fz.Operation(--extrinsicstate);
    UnsharedConcreteFlyweight uf = new UnsharedConcreteFlyweight();
    uf.Operation(--extrinsicstate);
}
}
/*Program Çalıştığında:
ConcreteFlyweight: 21
ConcreteFlyweight: 20
ConcreteFlyweight: 19
UnsharedConcreteFlyweight: 18

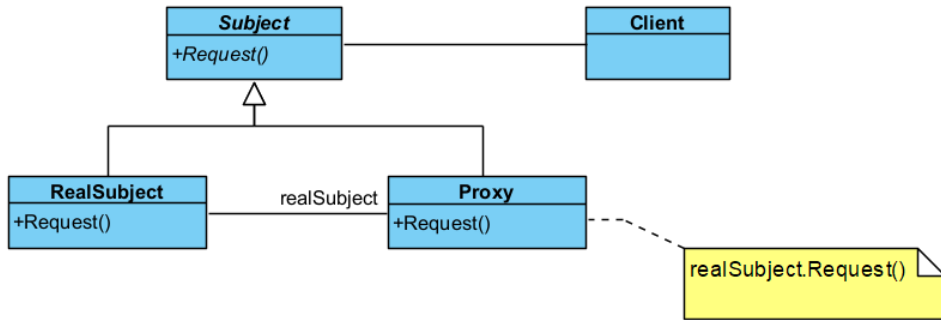
...Program finished with exit code 0
*/

```

Vekil Deseni

Vekil deseni (**proxy pattern**), kullanılacak ve erişilecek bir nesnenin yerine geçen bir başka nesneye ihtiyaç duyulduğunda kullanılır. **İstemci** (**client**), nesnenin doğrudan kendisine değil, **vekil** (**proxy**) nesneye ulaşır ve bu nesneye isteklerini iletir. Kısaca elinden her iş gelen veya diğer nesneler hakkında bilgi sahibi olan bir nesneye ihtiyaç duyulduğunda kullanılır.

- Bir nesne, bulunduğu yerden uzaktaki bir nesneyi kullanacaksa **uzak vekil** (**remote proxy**) kullanılır.
- Bir nesneye başka nesneler tarafından erişim kısıtlanacaksa **koruma vekili** (**protection proxy**) olarak adlandırılır.
- Bir nesneye erişim sırasında başka işlemler yapılacaksa vekil, **akıllı referans** (**smart reference**) olarak adlandırılır. Böyle durumda vekil, nesneye bir erişim yapıldığında, diğer nesnelerin erişmemesi için kullanılacak nesneye kilit koyabilir. Akıllı referans, **akıllı gösterici** (**smart pointer**) olarak da adlandırılır.



Şekil 45. Vekil Deseni UML Diyagramı

UML diyagramındaki **RealSubject**, bazı temel iş mantıklarını içerir. Genellikle, giriş verilerini düzeltme gibi hassas olabilen bazı yararlı işler yapma yeteneğine sahiptir. Bir **Proxy**, **RealSubject** kodunda herhangi bir değişiklik yapmadan bu sorunları çözebilir.

İstemci kodunun hem **RealSubject** hem de **Proxy** çalışabilmesi için tüm nesnelerle **Subject** ara yüzü üzerinden çalışması gerekir. Ancak gerçek hayatta, istemciler çoğunlukla **RealSubject** ile doğrudan çalışır. Bu durumda, deseni daha kolay uygulamak için **Proxy**, **RealSubject** sınıfından genişletebilirsiniz.

Şimdi **Subject** sanal sınıfı ve somut sınıflarını kodlayalım;

```

abstract class Subject
{
    public abstract void Request();
}
class RealSubject : Subject
{
    public override void Request()
    {
        Console.WriteLine("Called RealSubject.Request()");
    }
}

```

```

class Proxy : Subject
{
    RealSubject realSubject;
    public override void Request()
    {
        if (realSubject == null)
        {
            realSubject = new RealSubject();
        }
        realSubject.Request();
    }
}

```

Son olarak istemci kodunu kodlayalım;

```

class Client
{
    static void Main(string[] args)
    {
        Proxy proxy = new Proxy();
        proxy.Request();
    }
}
/* Program Çıktısı:
Called RealSubject.Request()

...Program finished with exit code 0
*/

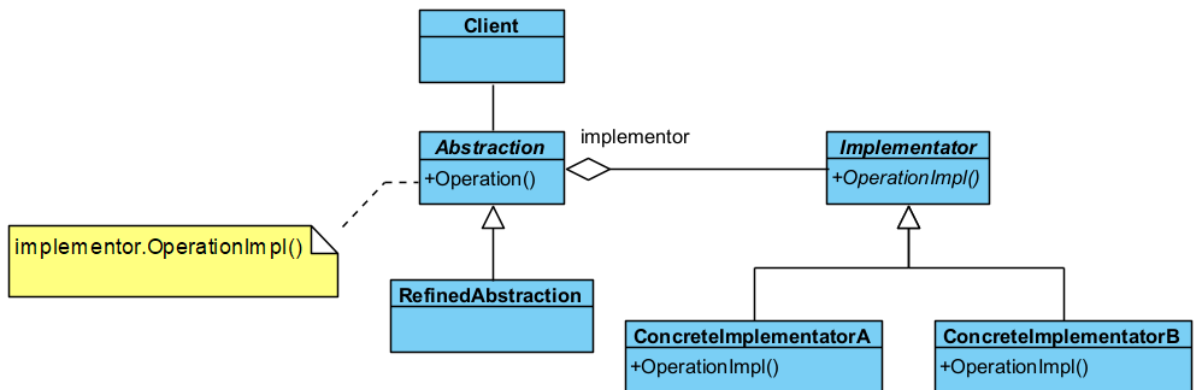
```

Köprü Deseni

Köprü deseni (bridge pattern), işi yapan alt yüklenici veya taşeron (implementator) nesne ile işi yaptıran nesne (client) arasındaki bağımlılığı soyutlama ile ortadan kaldırır. Yani birbiriyle oldukça iç içe olan kodlama ile soyutlamayı ortak ara yüz ile birbirinden uzaklaştırır. Sistemin genişlemesine izin verir.

“Bu desen, yazılımlarda oldukça çok kullanılan bir desendir. Çünkü taşeronlar değişse de yeni taşeron eklense de istemci tarafında bir şey değişmez. Köprü deseni aşağıdaki durumlarda kullanılır;

- Taşeron nesneye kalıcı bir bağımlılık istenmediği durumlar. Çalıştırma anında (run time) taşeronlar arasında geçiş yapmayı sağlar.
- Birden çok taşeron kullanarak sistemin genişlemesine ihtiyaç olduğu durumlar.
- Taşeronlar üzerindeki değişiklikler, istemciyi etkilemez. İstemci tarafında derleme olmadan yazılımın değiştirilmesi sağlanır.
- Projede aynı anda farklı birden çok taşeron tasarımı yapılabildiğinden, proje parçalara ayrılarak kendi içinde hızla sınıf tasarımı yapılabilir. Bu durum Rumbaugh tarafından iç içe genelleştirme (nested generalization) olarak adlandırılmıştır.



Şekil 46. Köprü Deseni UML Diyagramı

İlk önce taşeron olan soyut **Implementator** sınıfı ve somut alt sınıflarını tanımlayalım;

```

abstract class Implementator
{
    public abstract void OperationImpl();
}
class ConcreteImplementatorA : Implementator
{
    public override void OperationImpl()
    {
        Console.WriteLine("ConcreteImplementatorA İşlemi");
    }
}
class ConcreteImplementatorB : Implementator
{
    public override void OperationImpl()
    {
        Console.WriteLine("ConcreteImplementatorB İşlemi");
    }
}

```

Şimdi de yüklenici olan **Abstraction** sınıfını ve alt somut sınıfını kodlayalım;

```

abstract class Abstraction
{
    protected Implementator implementator;
    public Implementator Implementator
    {
        set { implementator = value; }
    }
    public virtual void Operation()
    {
        implementator.OperationImpl();
    }
}
class RefinedAbstraction : Abstraction
{
    public override void Operation()
    {
        implementator.OperationImpl();
    }
}

```

Son olarak istemi tarafını kodlayalım;

```

class Client
{
    static void Main(string[] args)
    {
        Abstraction ab = new RefinedAbstraction();
        // Set implementation and call
        ab.Implementator = new ConcreteImplementatorA();
        ab.Operation();
        ab.Implementator = new ConcreteImplementatorB();
        ab.Operation();
    }
}
/* Program Çıktısı:
ConcreteImplementatorA İşlemi
ConcreteImplementatorB İşlemi

...Program finished with exit code 0
*/

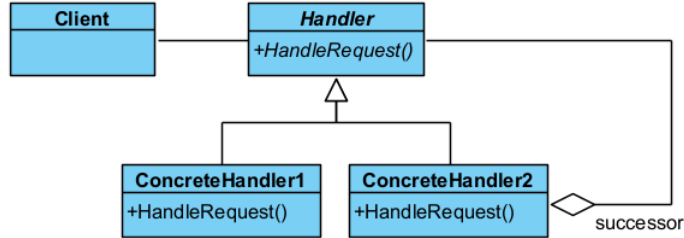
```

Davranışla İlgili Tasarım Desenleri

Davranışla ilgili desenler (behavioral patterns) nesnelerin çeşitli olaylar karşısında gösterecekleri davranışlara çözüm getirmektedirler.

Sorumluluk Zinciri Deseni

Bazı durumlarda bir nesne, kendinin yapmayacağı bir işlemi halefine (successor) devredebilir. Ya da nesnenin kendi sorumluluğunu aşan durumlarda ilgili diğer nesneye işlem yaptırılır. İşte bu durumda sorumluluk zinciri deseni (chain of responsibility pattern) kullanılır.



Şekil 47. Sorumluluk Zinciri Deseni UML Diyagramı

Şimdi soyut **Handler** sınıfı ve somut sınıflarını kodlayalım;

```

abstract class Handler
{
    public abstract void handleRequest(int pRequest);
}

class ConcreteHandler1: Handler
{
    public override void handleRequest(int pRequest)
    {
        Console.WriteLine("Yetki Devretmeyen ConcreteHandler1");
        Console.WriteLine("{0} ile gelen istek yerine getirildi!", pRequest);
    }
}

class ConcreteHandler2 : Handler
{
    private Handler successor;
    private int threshold = 10;
    public ConcreteHandler2(Handler pSuccessor)
    {
        successor = pSuccessor;
    }
    public override void handleRequest(int pRequest)
    {
        if (pRequest < threshold)
        {
            Console.WriteLine("Yetki Devretmeyen ConcreteHandler2");
            Console.WriteLine("{0} ile gelen istek yerine getirildi!", pRequest);
        }
        else
        {
            Console.WriteLine("ConcreteHandler2 Yetkisini Devretti! ");
            successor.handleRequest(pRequest);
        }
    }
}
  
```

Son olarak istemci kısmını kodlayalım;

```

class Client
{
    static void Main(string[] args)
  
```

```

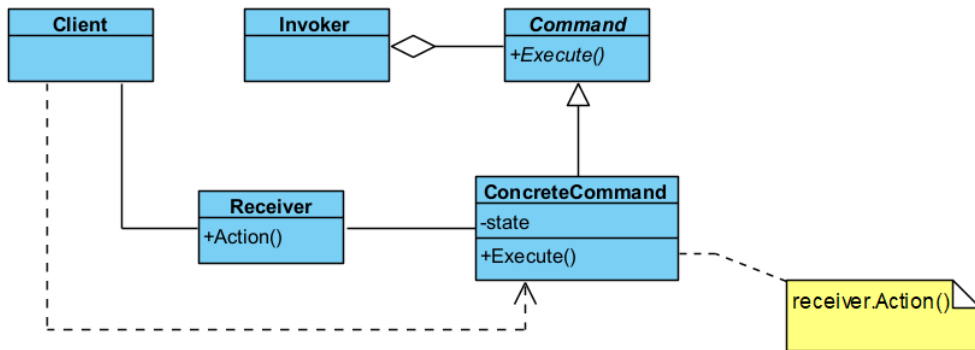
{
    int[] requests = { 5, 26, 10, 15, 9, 45 };
    Console.WriteLine("Requests:");
    foreach (int request in requests)
        Console.WriteLine(request + " ");
    Console.WriteLine();
    Handler müdür = new ConcreteHandler1(); //Yetkili
    müdür.handleRequest(5);
    müdür.handleRequest(20);
    Handler memur = new ConcreteHandler2(müdür); //Kısmi yetkili
    foreach (int request in requests)
        memur.handleRequest(request);
}
}
/* Program Çıktısı:
Requests:5 26 10 15 9 45
Yetki Devretmeyen ConcreteHandler1
5 ile gelen istek yerine getirildi!
Yetki Devretmeyen ConcreteHandler1
20 ile gelen istek yerine getirildi!
Yetki Devretmeyen ConcreteHandler2
5 ile gelen istek yerine getirildi!
ConcreteHandler2 Yetkisini Devretti!
Yetki Devretmeyen ConcreteHandler1
26 ile gelen istek yerine getirildi!
ConcreteHandler2 Yetkisini Devretti!
Yetki Devretmeyen ConcreteHandler1
10 ile gelen istek yerine getirildi!
ConcreteHandler2 Yetkisini Devretti!
Yetki Devretmeyen ConcreteHandler1
15 ile gelen istek yerine getirildi!
Yetki Devretmeyen ConcreteHandler2
9 ile gelen istek yerine getirildi!
ConcreteHandler2 Yetkisini Devretti!
Yetki Devretmeyen ConcreteHandler1
45 ile gelen istek yerine getirildi!

...Program finished with exit code 0
*/

```

Komut Deseni

Bazı durumlarda bir nesne diğer bir nesneye **ileti** (message) verip bir işlem başlattığında, bu işlem bitmeden başka işlemler yapmak isteyebilir. İşte bu tür durumlarda **komut deseni** (command pattern) kullanılır.



Şekil 48. Komut Deseni UML Diyagramı

Bu desende istemcinin istekleri, **Invoker** nesnesi tarafından kuyruğa sokulur ve kuyruğa sokulan adımların her biri **Command** nesnesi tarafından asıl işi yapan **Receiver** nesnesine yaptırılır. Bu durum bize **geri alma** (undo) işlemi yapmamızı sağlar.

İlk önce asıl işi yapacak **Receiver** sınıfını kodluyoruz;

```
class Receiver
{
    public void Action()
    {
        Console.WriteLine("Receiver.Action() İşlevi Çağrıldı");
    }
    public void Action(string pParam)
    {
        Console.WriteLine("Receiver.Action({0}) İşlevi Çağrıldı",pParam);
    }
    public void ActionA(string pParam)
    {
        Console.WriteLine("Receiver.ActionA({0}) İşlevi Çağrıldı", pParam);
    }
    public void ActionB(string pParam)
    {
        Console.WriteLine("Receiver.ActionB({0}) İşlevi Çağrıldı", pParam);
    }
}
```

Sonrasında her bir komutu tutacak soyut **Command** sınıfı ve somut sınıflarını yazıyoruz;

```
abstract class Command
{
    public abstract void Execute();
}
class ConcreteCommand : Command
{
    private Receiver receiver = new Receiver();
    private string state;

    public ConcreteCommand(string pState)
    {
        state = pState;
    }
    public override void Execute()
    {
        receiver.Action();
    }
}
class SimpleConcreteCommand: Command
{
    private Receiver receiver = new Receiver();
    private string state;
    public SimpleConcreteCommand(string pState)
    {
        state = pState;
    }
    public override void Execute() {
        receiver.Action(state);
    }
}
class ComplexConcreteCommand: Command
{
    private Receiver receiver = new Receiver();
    private string state;
    public ComplexConcreteCommand(string pState)
    {
        state = pState;
    }
    public override void Execute() {
```

```

        receiver.ActionA(state);
        receiver.Action();
        receiver.ActionB(state);
    }
}

```

Daha sonra ihtiyaç olursa birden fazla komut tutan ve çalıştıran **Invoker** sınıfını tanımlıyoruz;

```

class Invoker
{
    private List<Command> commands;
    public Invoker(List<Command> pList)
    {
        commands = pList;
    }
    public void addCommand(Command pCommand)
    {
        this.commands.Add(pCommand);
    }
    public void doCommands()
    {
        foreach (Command command in commands)
            command.Execute();
    }
}

```

Son olarak istemci kodunu yazıyoruz;

```

class Client
{
    static void Main(string[] args)
    {
        Command simpleCommand = new SimpleConcreteCommand("Yaz");
        simpleCommand.Execute();

        Command complexCommand = new ComplexConcreteCommand("Hem Dosyaya Hem Yazıcıya Yaz");

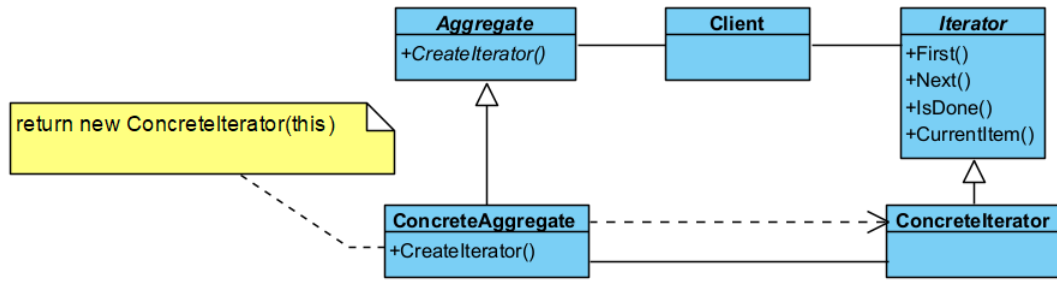
        List< Command> commandList = new List<Command>();
        commandList.Add(simpleCommand);
        commandList.Add(complexCommand);
        Invoker invoker = new Invoker(commandList);
        invoker.doCommands();
    }
}
/* Program çalıştığında;
Receiver.Action(Yaz) İşlevi Çağrıldı
Receiver.Action(Yaz) İşlevi Çağrıldı
Receiver.ActionA(Hem Dosyaya Hem Yazıcıya Yaz) İşlevi Çağrıldı
Receiver.Action() İşlevi Çağrıldı
Receiver.ActionB(Hem Dosyaya Hem Yazıcıya Yaz) İşlevi Çağrıldı

...Program finished with exit code 0
*/

```

Yineleyici Deseni

Yineleyici deseninde (*iterator pattern*), birden fazla elemandan oluşan küme (*aggregate*) içindeki her bir elemana sırayla erişim (*iteration*) sağlar. Erişim işleminde, istemcinin kümenin nasıl yapılandırıldığı konusunda bilgi sahibi olması gerekmez. Bu desende, istemci tarafından veri kümesi soyut (*abstract*) olarak kullanılmış olur. Yani istemciye, veri yapısından (*data structure*) bağımsız bir şekilde verilere erişim sağlayan bir ara yüz (*interface*) sunulur.



Şekil 49. Yineleyici Deseni UML Diyagramı

Burada **Aggregate** ve **Iterator** sınıflarını ilk önce tanımlıyoruz;

```

abstract class Aggregate
{
    public abstract Iterator CreateIterator();
}
class ConcreteAggregate : Aggregate
{
    private ArrayList items = new ArrayList();
    public override Iterator CreateIterator()
    {
        return new ConcreteIterator(this);
    }
    public int Count
    {
        get { return items.Count; }
    }
    public object this[int index]
    {
        get { return items[index]; }
        set { items.Insert(index, value); }
    }
}
abstract class Iterator
{
    public abstract object First();
    public abstract object Next();
    public abstract bool IsDone();
    public abstract object CurrentItem();
}
class ConcreteIterator : Iterator
{
    private ConcreteAggregate aggregate;
    private int current = 0;
    public ConcreteIterator(ConcreteAggregate aggregate)
    {
        this.aggregate = aggregate;
    }
    public override object First()
    {
        return aggregate[0];
    }
    public override object Next()
    {
        object ret = null;
        if (current < aggregate.Count - 1)
        {
            ret = aggregate[++current];
        }
        return ret;
    }
    public override object CurrentItem()
    
```

```

    {
        return aggregate[current];
    }
    public override bool IsDone()
    {
        return current >= aggregate.Count ? true : false;
    }
}

```

Şimdi de istemci kısmını kodluyoruz;

```

class Client
{
    static void Main(string[] args)
    {
        ConcreteAggregate a = new ConcreteAggregate();
        a[0] = "Item A";
        a[1] = "Item B";
        a[2] = "Item C";
        a[3] = "Item D";
        ConcreteIterator i = new ConcreteIterator(a);
        // Burada Yığın birden çok nesneden oluşan heshangi bir
        // nesneler topluluğudur.
        Console.WriteLine(
            "Yığın (collection) üzerinde yineleme:");
        // Yineleyici, yığın hakkında bilgi sahibi olmadan
        // elemanlar üzerinde işlem yapmamızı sağlar.
        // Bir başka deyişle kullanıcı yığına bağlı kalmaz.
        object item = i.First();
        while (item != null)
        {
            Console.WriteLine(item);
            item = i.Next();
        }
    }
}
/*
Yığın (collection) üzerinde yineleme:
Item A
Item B
Item C
Item D

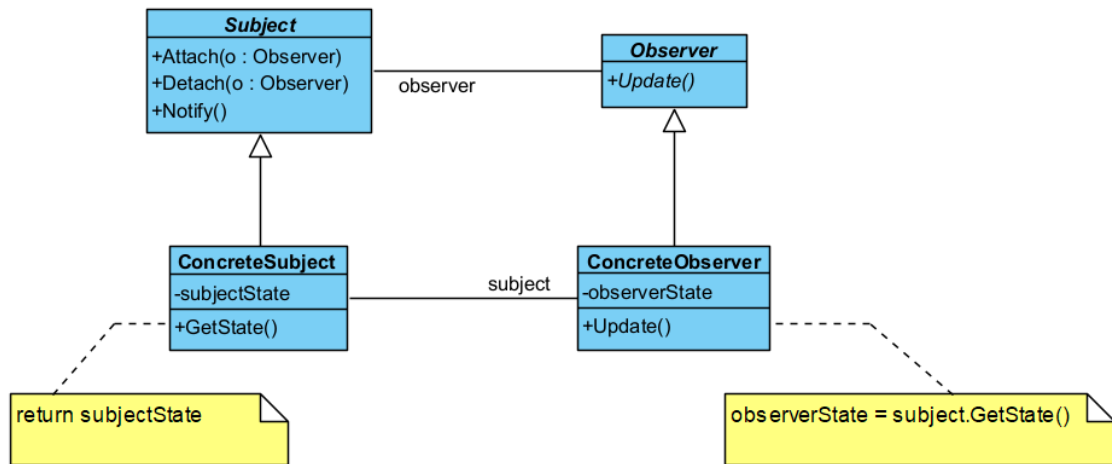
...Program finished with exit code 0
*/

```

Gözlemci Deseni

Gözlemci deseni (**observer pattern**), sistemdeki diğer nesnelerin durum değişiklikleri hakkında bir veya daha fazla nesnenin bilgilendirilmesini sağlar. Bağımlı olunan nesnenin durumu veya işin **konusu** (**subject**) değiştiğinde, bağımlı olan veya o işi izleyen **gözlemcilerin** (**observer**) kendileri de güncellenir. Bu desen aşağıdaki durumlarda kullanılır;

- Bir **soyutlama** (**abstraction**) sonucu ortaya çıkan bağımsız iki **farklı yönün** (**aspect**) ortaya çıktığı durumlarda, bu yönler birbirinden bağımsız olarak geliştirilebilir ve değiştirilebilir.
- Bir değişikliğin başka değişiklikleri gerektirdiği durumlar.
- Bir nesnenin, diğer nesnelerdeki değişiklikleri kendine vazife etmemesini sağlayan durumlar.



Şekil 50. Gözlemci Deseni UML Diyagramı

İlk önce sanal **Observer** sınıfını tanımlayalım;

```
abstract class Observer
{
    public abstract void Update();
}
```

Daha sonra sanal **Subject** ve somut **ConcreteSubject** sınıfını kodlayalım;

```
abstract class Subject
{
    private ArrayList observers = new ArrayList();
    public void Attach(Observer observer)
    {
        observers.Add(observer);
    }
    public void Detach(Observer observer)
    {
        observers.Remove(observer);
    }
    public void Notify()
    {
        foreach (Observer o in observers)
        {
            o.Update();
        }
    }
}
// "ConcreteSubject"
class ConcreteSubject : Subject
{
    private string subjectState;
    public string SubjectState
    {
        get { return subjectState; }
        set { subjectState = value; }
    }
}
```

Daha sonra somut **ConcreteObserver** Sınıfını tanımlayalım;

```
class ConcreteObserver : Observer
{
    private string name;
    private string observerState;
    private ConcreteSubject subject;
```

```

public ConcreteObserver(ConcreteSubject subject,
    string name)
{
    this.subject = subject;
    this.name = name;
}
public override void Update()
{
    observerState = subject.SubjectState;
    Console.WriteLine(
        "Observer {0}'s new state is {1}", name,
        observerState);
}
public ConcreteSubject Subject
{
    get { return subject; }
    set { subject = value; }
}
}

```

Son olarak istemci tarafını kodlayalım;

```

class Client
{
    static void Main(string[] args)
    {
        ConcreteSubject s = new ConcreteSubject();
        s.Attach(new ConcreteObserver(s, "X"));
        s.Attach(new ConcreteObserver(s, "Y"));
        s.Attach(new ConcreteObserver(s, "Z"));
        s.SubjectState = "ABC";
        s.Notify();
    }
}
/*Program Çıktısı:
Observer X's new state is ABC
Observer Y's new state is ABC
Observer Z's new state is ABC

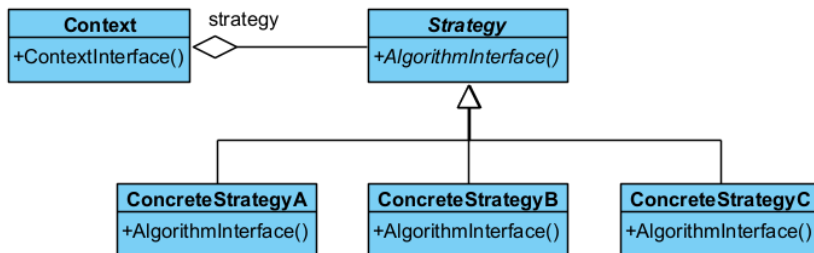
...Program finished with exit code 0
*/

```

Strateji Deseni

Strateji deseni (**strategy pattern**), belirli bir davranışı gerçekleştirmek için değiştirilebilen **sarmalanmış** (**encapsulated**) algoritmalar kümesini tanımlar. Bu desen aşağıdaki durumda kullanılır;

- Aralarında ilişki bulunan birçok sınıfın davranışlara bağlı olarak anlaşmazlığa düşmesini önlemek için,
- Birden çok algoritmanın olması durumunda,
- Algoritma, istemcinin bilmeyeceği bir veri yapısına sahip olduğu durumlarda,
- Bir sınıf kendi yöntemlerinde çok fazla sayıda **duruma göre seçim** (**conditional choice**) kullanıyorsa.



Şekil 51. Strateji Deseni UML Diyagramı

Bu desenin **bağlam** (**Context**) nesnesi, diğer tasarım desenlerinin biri ile iç içe kullanılması halinde, **sıfır desen** (**null pattern**) olarak adlandırılır. Sıfır desende, **Context** nesnesi akıllıdır ve her zaman ne yapacağını bilir.

İlk önce **Strategy** somut sınıfı ve alt sınıflarını kodlayalım;

```
abstract class Strategy
{
    public abstract void AlgorithmInterface();
}
class ConcreteStrategyA : Strategy
{
    public override void AlgorithmInterface()
    {
        Console.WriteLine(
            "ConcreteStrategyA.AlgorithmInterface() Çağrıldı.");
    }
}
class ConcreteStrategyB : Strategy
{
    public override void AlgorithmInterface()
    {
        Console.WriteLine(
            "ConcreteStrategyB.AlgorithmInterface() Çağrıldı.");
    }
}
class ConcreteStrategyC : Strategy
{
    public override void AlgorithmInterface()
    {
        Console.WriteLine(
            "ConcreteStrategyC.AlgorithmInterface() Çağrıldı.");
    }
}
```

Şimdi de **Context** sınıfını kodlayalım;

```
class Context
{
    Strategy strategy;
    public Context(Strategy strategy)
    {
        this.strategy = strategy;
    }
    public void ContextInterface()
    {
        strategy.AlgorithmInterface();
    }
}
```

Son olarak istemciyi kodlayalım;

```
class Client
{
    static void Main(string[] args)
    {
        Context context;
        // Three contexts following different strategies
        context = new Context(new ConcreteStrategyA());
        context.ContextInterface();
        context = new Context(new ConcreteStrategyB());
        context.ContextInterface();
        context = new Context(new ConcreteStrategyC());
        context.ContextInterface();
    }
}
```

```

    }
}
/*Program Çıktısı:
ConcreteStrategyA.AlgorithmInterface() Çağrıldı.
ConcreteStrategyB.AlgorithmInterface() Çağrıldı.
ConcreteStrategyC.AlgorithmInterface() Çağrıldı.

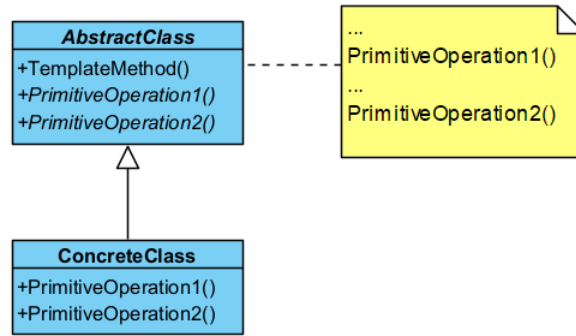
...Program finished with exit code 0
*/

```

Şablon Yöntem Deseni

Şablon yöntem deseninde (template method pattern), bir algoritmanın çerçevesini belirler ve uygulayıcı sınıfların gerçek davranışı tanımlamasına olanak tanır. Kullanılacak algoritmalarından hangilerinin standart hangilerinin somut sınıflara özgü olacağı belirlenmelidir;

- İçinde “Bizi çağırma, biz sizi çağıracağız” sloganını taşıyan yöntemler olan bir soyut bir taban sınıf tasarlanmalıdır.
- Taban sınıf içinde algoritmanın kabuğunu oluşturacak standart adımları içeren şablon yöntem tanımlanır.
- Somut sınıflarda detaylandırılacak yöntemlere ilişkin sanal yöntemler taban sınıfta tanımlanır.
- Şablon yöntem içinde algoritmayı oluşturan yöntemler çağrılır.
- Şablon yöntem içinde yer almayan tüm detay kodlamalar somut sınıflar için de yapılır.



Şekil 52. Şablon Yöntem Deseni UML Diyagramı

Şimdi Programı kodlayalım;

```

abstract class AbstractClass
{
    public abstract void PrimitiveOperation1();
    public abstract void PrimitiveOperation2();
    // The "Template method"
    public void TemplateMethod()
    {
        PrimitiveOperation1();
        PrimitiveOperation2();
    }
}
// "ConcreteClass"
class ConcreteClassA : AbstractClass
{
    public override void PrimitiveOperation1()
    {
        Console.WriteLine(
            "ConcreteClassA.PrimitiveOperation1()");
    }
    public override void PrimitiveOperation2()
    {
        Console.WriteLine(
            "ConcreteClassA.PrimitiveOperation2()");
    }
}

```

```

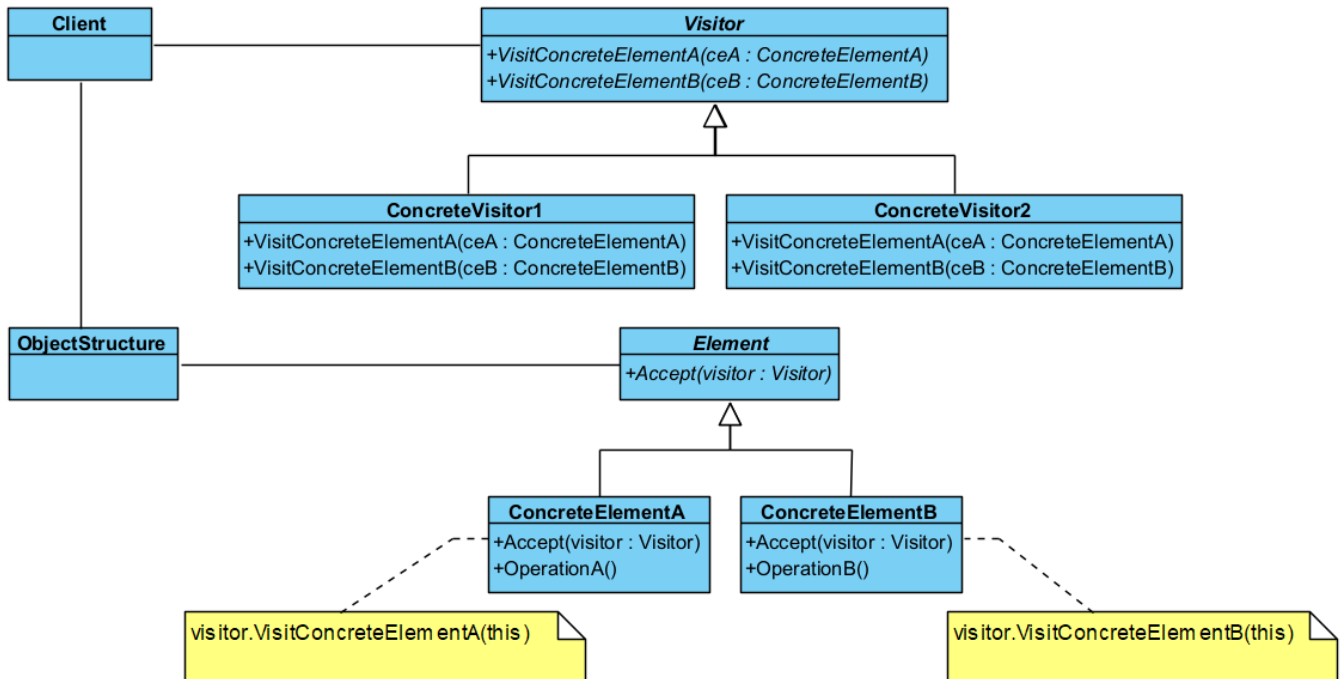
}
class ConcreteClassB : AbstractClass
{
    public override void PrimitiveOperation1()
    {
        Console.WriteLine(
            "ConcreteClassB.PrimitiveOperation1()");
    }
    public override void PrimitiveOperation2()
    {
        Console.WriteLine(
            "ConcreteClassB.PrimitiveOperation2()");
    }
}
}
class Client
{
    static void Main(string[] args)
    {
        AbstractClass c;
        c = new ConcreteClassA();
        c.TemplateMethod();
        c = new ConcreteClassB();
        c.TemplateMethod();
    }
}
/*Program Çıktısı:
ConcreteClassA.PrimitiveOperation1()
ConcreteClassA.PrimitiveOperation2()
ConcreteClassB.PrimitiveOperation1()
ConcreteClassB.PrimitiveOperation2()

...Program finished with exit code 0
*/

```

Ziyaretçi Deseni

Ziyaretçi deseni (visitor pattern), bir sınıfa ilişkin verileri kullanarak işlem yapan bir başka yeni sınıf tanımlanmak istendiğinde kullanılır. Yani çalışma zamanında bir veya daha fazla işlemin bir nesne kümesine uygulanmasına izin verir ve işlemleri nesne yapısından ayırır.



Şekil 53. Ziyaretçi Deseni UML Diyagramı

Başka amaçla tanımlanan yeni sınıfın (**Visitor**), mevcut sınıfın (**Element**) verilerine ulaşmak istemesi halinde bu desen kullanılır. Bu desen aşağıdaki üstünlükleri sağlar;

- Yeni işlemleri (**operation**) sisteme eklemeyi kolaylaştırır.
- Birbiriyle ilişkili olan işlemleri ilişkili olmayandan ayırmaya yardımcı olur.
- Yeni **ConcreteElement** nesnelerinin eklenmesini zorlaştırır.
- Bütün nesne hiyerarşisi baştanbaşa ziyaret edilebilir.
- Sınıflardaki tüm durumlar (**state**) biriktirilebilir.
- Bazı durumlarda **sarma** (**encapsulation**) işlemini engeller. Yani nesnenin iç durumlarından hareketle herkese açık işlemler tanımlanabilir.

Bu desen aşağıdaki durumlarda kullanılır;

- Nesneler birden fazla olup birçok ara yüze sahip olduğunda, bu nesnelerin somut sınıfları ile bir işlem yapmak gerektiğinde,
- Sınıflar üzerinde yapılacak işlemlerin, sınıflar üzerinde bir iz bırakmamasını sağlamak amacıyla,
- Nadir değişen sınıf yapılarına, sınıf yapısını değiştirmeden, yeni işlemler eklemek gerektiğinde.

İlk önce **Visitor** sınıflarını kodlayalım;

```
abstract class Visitor
{
    public abstract void VisitConcreteElementA(ConcreteElementA concreteElementA);
    public abstract void VisitConcreteElementB(ConcreteElementB concreteElementB);
}
class ConcreteVisitor1 : Visitor
{
    public override void VisitConcreteElementA( ConcreteElementA concreteElementA)
    {
        Console.WriteLine("{0} visited by {1}",
            concreteElementA.GetType().Name,
            this.GetType().Name);
    }
    public override void VisitConcreteElementB(ConcreteElementB concreteElementB)
    {
        Console.WriteLine("{0} visited by {1}",
            concreteElementB.GetType().Name,
            this.GetType().Name);
    }
}
class ConcreteVisitor2 : Visitor
{
    public override void VisitConcreteElementA(ConcreteElementA concreteElementA)
    {
        Console.WriteLine("{0} visited by {1}",
            concreteElementA.GetType().Name,
            this.GetType().Name);
    }
    public override void VisitConcreteElementB(ConcreteElementB concreteElementB)
    {
        Console.WriteLine("{0} visited by {1}",
            concreteElementB.GetType().Name,
            this.GetType().Name);
    }
}
```

Daha sonra **Element** sınıflarını kodlayalım;

```
abstract class Element
{
    public abstract void Accept(Visitor visitor);
}
class ConcreteElementA : Element
{
    }
```



```
    public override void Accept(Visitor visitor)
    {
        visitor.VisitConcreteElementA(this);
    }
    public void OperationA() { }
}
class ConcreteElementB : Element
{
    public override void Accept(Visitor visitor)
    {
        visitor.VisitConcreteElementB(this);
    }
    public void OperationB() { }
}
```

Bir de **ObjectStructure** sınıfını kodlayalım;

```
class ObjectStructure
{
    private ArrayList elements = new ArrayList();
    public void Attach(Element element)
    {
        elements.Add(element);
    }
    public void Detach(Element element)
    {
        elements.Remove(element);
    }
    public void Accept(Visitor visitor)
    {
        foreach (Element e in elements)
        {
            e.Accept(visitor);
        }
    }
}
```

Son olarak istemci kodumuzu kodlayalım;

```
class Client
{
    static void Main(string[] args)
    {
        // Setup structure
        ObjectStructure o = new ObjectStructure();
        o.Attach(new ConcreteElementA());
        o.Attach(new ConcreteElementB());
        // Create visitor objects
        ConcreteVisitor1 v1 = new ConcreteVisitor1();
        ConcreteVisitor2 v2 = new ConcreteVisitor2();
        // Structure accepting visitors
        o.Accept(v1);
        o.Accept(v2);
    }
}
/* Programın Çıktısı:
ConcreteElementA visited by ConcreteVisitor1
ConcreteElementB visited by ConcreteVisitor1
ConcreteElementA visited by ConcreteVisitor2
ConcreteElementB visited by ConcreteVisitor2

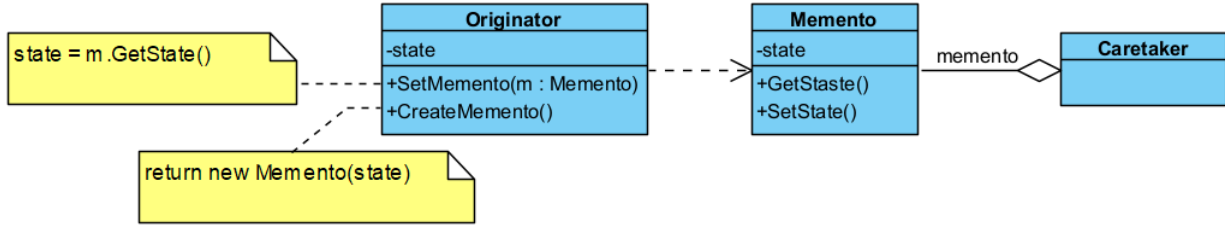
...Program finished with exit code 0
*/
```

Ziyaretçi deseni uygulanırken iki kodlama (**implementation**) sorunu irdelenmelidir;

- **Çifte sevkiyat (double dispatch)**, Bir sınıfı hiçbir şekilde değiştirmeden, söz konusu sınıfa etkin biçimde yöntem eklenebilmesi işlemidir. Bu çok bilinen bir tekniktir ve **Common Lisp Object System (CLOS)** gibi programlama dilleri tarafından desteklenir ve nesne üzerinde sarma (**encapsulation**) yapılmasına izin verilmez.
- C++, C#, Java gibi birçok nesne yönelimli programlama dili **tek sevkiyat (single dispatch)** yöntemi ile çalışır. Bu teknik ziyaretçi deseni için anahtardır. **Accept** yöntemi çifte sevkiyat sağlayan yöntemdir. Yöntemler, statik olarak yalnızca **Element** sınıfına bağlanmaz. **Element** sınıfı yapılacak işlemleri **Visitor** üzerinde birleştirir ve **Accept** yöntemi ile çalıştırma anında icra eder. İcra edilen yöntemler, **Visitor** ve **Element** veri tipi olmak üzere iki veri tipine bağlıdır.
- İşlemler dışarıda tutularak oluşturulan nesne yapısının sorumlusu kimdir? **Visitor**, oluşturulan nesne yapısının her bir elemanını tek tek ziyaret etmek zorundadır. Bunun nasıl ve hangi sırada yapılacağı belirgin değildir. Birçok durumda nesne yapısının kendisi bundan sorumlu tutulur. Nesne yapısı basit bir küme olarak tanımlanır ve **Accept** yöntemi her biri için çağrılır. Bazı durumlarda da **yineleyici deseni (iterator pattern)** kullanılır.

Hatıra Deseni

Hatıra deseni (memento pattern), nesnelerin bir andaki **durumlarını (state)** saklayıp bir başka zaman da bu duruma geri dönmek gerektiğinde kullanılır. Bir nesnenin iç durumunun yakalanmasına ve dışarda saklanmasına olanak tanır, böylece daha sonra geri yüklenebilir fakat tüm bunlar **sarmalamayı (encapsulation)** ihlal etmeden yapılır.



Şekil 54. Hatıra Deseni UML Diyagramı

Yukarıdaki diyagram incelendiğinde; **Originator** nesnesi, geçmişteki duruma dönecek nesneyi, **Memento** ise **Originator** nesnesinin bir anlık resmini çeken nesneyi, **Caretaker** ise **Memento** nesnelerini saklayan yardımcı nesneyi ifade etmektedir.

İlk olarak **Memento** sınıfını kodlayalım;

```
class Memento
{
    private string state;
    public Memento(string state)
    {
        this.state = state;
    }
    public string State
    {
        get { return state; }
    }
}
```

Daha sonra **Caretaker** sınıfını kodlayalım;

```
class Caretaker
{
    private Memento memento;
    public Memento Memento
    {
        set { memento = value; }
        get { return memento; }
    }
}
```

```
}
```

Bir de **Originator** sınıfını kodlayalım;

```
class Originator
{
    private string state;
    // Property
    public string State
    {
        get { return state; }
        set
        {
            state = value;
            Console.WriteLine("State = " + state);
        }
    }
    public Memento CreateMemento()
    {
        return (new Memento(state));
    }
    public void SetMemento(Memento memento)
    {
        Console.WriteLine("Restoring state:");
        State = memento.State;
    }
}
```

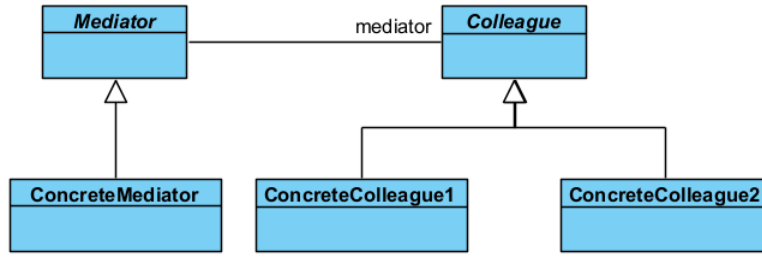
Son olarak istemci kodunu yazalım;

```
class Client
{
    static void Main(string[] args)
    {
        Originator o = new Originator();
        o.State = "On";
        // Store internal state
        Caretaker c = new Caretaker();
        c.Memento = o.CreateMemento();
        // Continue changing originator
        o.State = "Off";
        // Restore saved state
        o.SetMemento(c.Memento);
    }
}
/*Program Çıktısı:
State = On
State = Off
Restoring state:
State = On

...Program finished with exit code 0
*/
```

Arabulucu Deseni

Arabulucu deseni (**mediator pattern**), sınıfların arasındaki iletişimi basitleştirir. Bir iletişim nesnesi tanımlanır, diğer nesneler bu nesne üzerinden birbirleriyle haberleşir. Diğer nesnelerin birbiriyle doğrudan iletişim kurmasına izin verilmez. Amaç, iletişimin kontrol altına alınmasıdır. Bu desende arabulucunun (**mediator**) haberi olmadan hiçbir kişi (**colleauge**) birbiriyle haberleşmez. Ya da bir başka deyişle arabulucunun haberi olmadan hiç kimse birbiriyle görüşmez.



Şekil 55. Arabulucu Deseni UML Diyagramı

Bu desende, sistemde yalnızca bir adet arabulucu olacaksa soyut **Mediator** nesnesi tanımlanmaz. Ayrıca arabulucu ile kişiler arasındaki iletişim iki türlü yapılır;

- Birincisinde bu iletişim **gözlemci deseni (observer pattern)** ile sağlanır. **Colleague** sınıfı gözlemci deseniindeki **Subject** sınıf gibi davranır.
- İkincisinde ise **Mediator** nesnesine, **kişiler (colleage)** görüşecekleri kişileri belirterek doğrudan ileti gönderirler.

İlk olarak **Mediator** sınıfını kodlayalım;

```

abstract class Mediator
{
    protected List<Colleague> colleagues = new List<Colleague>();
    public abstract void Attach(Colleague colleague);
    public abstract void Send(string message, Colleague colleague);
}
class ConcreteMediator : Mediator
{
    public override void Attach(Colleague colleague)
    {
        colleagues.Add(colleague);
    }
    public override void Send(string message, Colleague colleague)
    {
        Console.WriteLine("LOG: {0}, {1} Mesajını Gönderdi!", colleague.Name, message);

        foreach (Colleague college in colleagues)
            college.GetMessage(message);
    }
}
  
```

Daha Sonra **Colleague** sınıfını kodlayalım;

```

abstract class Colleague
{
    protected Mediator mediator;
    public string Name { get; set; }
    public Colleague(string name, Mediator mediator)
    {
        this.mediator = mediator;
        Name = name;
        mediator.Attach(this);
    }
    public abstract void SendMessage(string message);
    public abstract void GetMessage(string message);
}
class ConcreteColleague : Colleague
{
    public ConcreteColleague(string name, Mediator mediator) : base(name, mediator)
    {
        Name = name;
    }
}
  
```

```

public override void SendMessage(string message)
{
    mediator.Send(message, this);
}
public override void GetMessage(string message)
{
    Console.WriteLine("{0} gets message: {1}", Name, message);
}
}

```

Son olarak istemci sınıfını kodlayalım;

```

class Client
{
    static void Main(string[] args)
    {
        Mediator arabulucu = new ConcreteMediator();
        Colleague ilhan = new ConcreteColleague("İlhan", arabulucu);
        Colleague mehmet = new ConcreteColleague("Mehmet", arabulucu);
        Colleague abdullah = new ConcreteColleague("Abdullah", arabulucu);

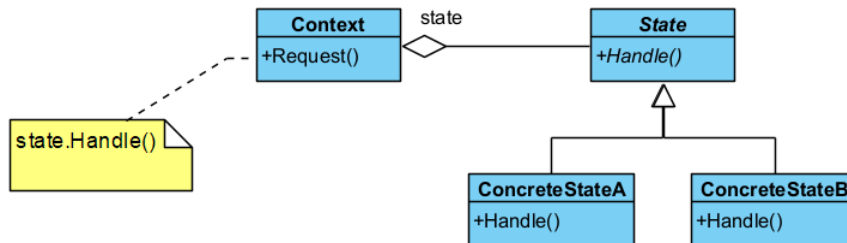
        ilhan.SendMessage("How are you?");
        mehmet.SendMessage("Fine, thanks");
        abdullah.SendMessage("I am good too");
    }
}
/* Program Çıktısı:
LOG: İlhan, How are you? Mesajını Gönderdi!
İlhan gets message: How are you?
Mehmet gets message: How are you?
Abdullah gets message: How are you?
LOG: Mehmet, Fine, thanks Mesajını Gönderdi!
İlhan gets message: Fine, thanks
Mehmet gets message: Fine, thanks
Abdullah gets message: Fine, thanks
LOG: Abdullah, I am good too Mesajını Gönderdi!
İlhan gets message: I am good too
Mehmet gets message: I am good too
Abdullah gets message: I am good too

...Program finished with exit code 0
*/

```

Durum Deseni

Durum deseni (**state pattern**), nesnenin durumunu davranışına bağlar, nesnenin içsel durumuna göre farklı şekillerde davranmasına olanak tanır.



Şekil 56. Durum Deseni UML Diyagramı

Bu desenin bağlam (**Context**) nesnesinin, diğer tasarım desenleri ile iç içe kullanılması halinde, **sıfır desen** (**null pattern**) olarak adlandırılır. Bu desenle ilgili olarak iki önemli şeyden bahsetmek gerekir:

- Sahip olunan duruma göre icra edilecek davranış, tanımlanan **State** sınıfına bağlıdır. Yani ilgili davranış somut sınıflardan biri tarafından icra edilir. Bu nedenle somut **State** sınıfları içinde tanımlanan yöntemler durumu kullanan bağlam (**Context**) sınıf içinde tanımlanmak zorundadır.

- Durumu kullanan bağlam (**Context**) sınıf içinde hangi duruma geçildiğinin anlaşılması için ilgili duruma ilişkin tanımlanan **özellik** (**property**) içinde set ifadesi kullanılmak zorundadır.

Bu desen netice olarak, nesnenin duruma özgü davranış göstermesini sağlar veya farklı durumlara göre davranışları birbirinden ayırır, **durum geçişlerini** (**state transition**) açığa çıkarır. Bu deseni kullanırken aşağıda verilen durumlar özellikle irdelenmelidir;

- Durum geçişleri kim tarafından tanımlanmalıdır? Bu desen, hangi şartlarda durumların ayrıştırılacağını belirtmez. Eğer şartlar belirgin ise bu işlem **Context** nesnesi tarafından yapılır. Bu şartlar belirgin değil ve karmaşık ise bu işlem **State** nesnesi ve **halefi** (**successor**) tarafından yapılır.
- Tablo tabanlı alternatif: Bu durumda durum geçişleri bir tablo aracılığı ile yapılır. Tablo üzerine sağlanacak her bir durum yazılır. Bu durumda, durum geçişlerini sağlayacak değişiklikler yeniden kodlama gerektirmez. Ancak bu yöntemde hız düşer. Durum geçişleri **tekdüze** (**uniform**) tanımlanmak zorundadır.
- **State** nesnelerini yaratma ve yok etme: Bu nesneler çalıştırma anında yaratılıp yok edildiklerinden, bu süre zarfında yapılacak işlemler için bir çözüm bulunmalıdır. İki yaklaşım vardır. Birincisi, **State** nesnesini ihtiyaç olduğunda yaratmak kullanmak ve öldürmektir. İkincisi ise bu nesneleri baştan yaratıp hiç öldürmemektir. Genellikle birinci yöntem tercih edilir.
- **Dinamik kalıtım** (**dynamic inheritance**) kullanma: Bazı durumlarda nesne, ait olduğu sınıfı çalıştırma anında değiştirir. Bu durumda talep edilen isteğin yerine getirilmesi tam olarak başarılabilir. Ancak birçok nesne yönelimli programlama dili bunu desteklemez.

Aşağıda durum nesnelerini değiştiren basit bir örnek verilmiştir;

```
class Context
{
    private State state,state1,state2;

    public Context(State state1, State state2)
    {
        this.state1 = state1;
        this.state2 = state2;
        state = state1;
    }

    private void durumDegistir()
    {
        if (state == state1)
            state = state2;
        else
            state = state1;
        Console.WriteLine("Context: Durum nesnesi değişti!");
    }
    public void Request()
    {
        state.Handle();
        durumDegistir();
    }
}
abstract class State
{
    public abstract void Handle();
}
class ConcreteStateA : State
{
    public override void Handle()
    {
        Console.WriteLine("ConcreteStateA: Handle method called.");
    }
}
class ConcreteStateB : State
```

```

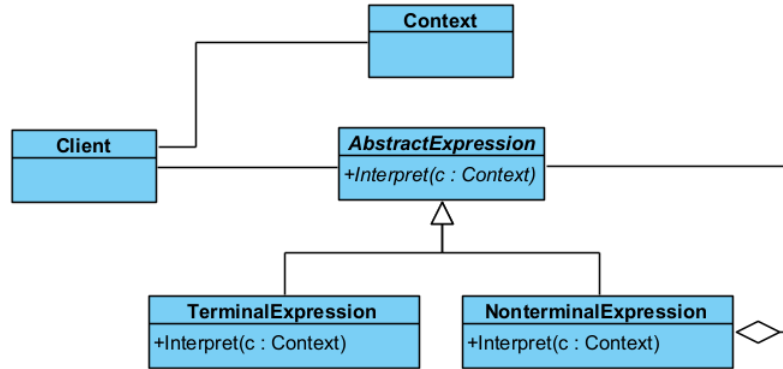
{
    public override void Handle()
    {
        Console.WriteLine("ConcreteStateB: Handle method called.");
    }
}
}
class Client
{
    static void Main(string[] args)
    {
        State durumA = new ConcreteStateA();
        State durumB = new ConcreteStateB();
        Context context = new Context(durumA, durumB);
        context.Request();
        context.Request();
        context.Request();
    }
}
}
/* Program Çıktısı:
ConcreteStateA: Handle method called.
Context: Durum nesnesi değişti!
ConcreteStateB: Handle method called.
Context: Durum nesnesi değişti!
ConcreteStateA: Handle method called.
Context: Durum nesnesi değişti!

...Program finished with exit code 0
*/

```

Yorumlayıcı Deseni

Yorumlayıcı deseni (interpreter pattern), programlama dili yorumlama özelliğini uygulamalarımıza katmanın yolunu gösterir. Yani bir dilbilgisi için bir temsili ve aynı zamanda dilbilgisini anlayıp ona göre hareket etmeyi sağlayacak bir mekanizmayı tanımlar.



Şekil 57. Yorumlayıcı deseni UML Diyagramı

Bu desen, yorumlanacak dilin dilbilgisi (grammar) kurallarının basit olduğu durumlarda kullanılır. Karmaşık dilbilgisi kurallarına sahip diller için yaratılacak sınıfların yönetimi oldukça zorlaşır. Burada, bağlam (Context) nesnesi dilbilgisi kurallarına sahip yorumlanacak nesneyi gösterir. TerminalExpression, en ilkel dil ögesini yorumlayacak nesnedir. NonterminalExpression ise ilkel dil öğelerinden oluşacak karmaşık yapıyı yorumlayacak nesneyi belirtir.

Bu deseni uygularken aşağıda verilen durumlar irdelenmelidir;

- Soyut sözdizimin oluşturulması: Bu desen yorumlanacak dile ilişkin soyut bir sözdizimin nasıl olması gerektiğini açıklamaz. Tabloya dayalı, ağaç yapısı şeklinde el becerisi ile veya doğrudan istemci tarafından yapılabilir.
- Yorumlayacak işlemin tanımlanması: Eğer işlemler ortak bir yapıdaysa yeni bir Expression nesnesi tanımlanarak yapılabilir. Daha karmaşık durumlarda ziyaretçi deseni kullanılır. Programlama

dillerinin çoğu soyut bir sözdizimine sahiptir ve yorumlanacak öğeler ayrı bir **Visitor** nesnesi tarafından yorumlanır.

- Cümle sonlarını bitiren ortak semboller için sineksiklet deseni kullanılabilir: Cümle sonlarındaki semboller genellikle bilgi saklamazlar. Yorumlamaya gerek duyulmazlar. Bu nedenle **sineksiklet deseni** (**flyweight pattern**) yorumlanacak yerler için bir eleme yapar.

Aşağıda kavramsal olarak bu desenin kolanmış örneği bulunmaktadır;

```
class Context
{
    private string context;
    public Context(string pContext)
    {
        this.context = pContext;
    }
}

abstract class AbstractExpression
{
    public abstract void interpret(Context context);
}

class TerminalExpression : AbstractExpression
{
    public override void interpret(Context context)
    {
        Console.WriteLine("Terminal Expression İşlemi Yorumluyor...");
    }
};

class NonterminalExpression : AbstractExpression
{
    private AbstractExpression abstractexpression;
    public override void interpret(Context context)
    {
        Console.WriteLine("Non Terminal Expression İşlemi Yorumluyor...");
        abstractexpression = new TerminalExpression();
        abstractexpression.interpret(context);
    }
}

class Client
{
    static void Main(string[] args)
    {
        List<AbstractExpression> expressions=new List<AbstractExpression>();
        AbstractExpression nte = new NonterminalExpression();
        AbstractExpression te = new TerminalExpression();
        expressions.Add(te);
        expressions.Add(nte);
        Context context = new Context("987698");
        foreach (AbstractExpression ae in expressions)
            ae.interpret(context);
    }
}

/*Program Çıktısı:
Terminal Expression İşlemi Yorumluyor...
Non Terminal Expression İşlemi Yorumluyor...
Terminal Expression İşlemi Yorumluyor...

...Program finished with exit code 0
*/
```


Model-View-Controller Mimari Deseni

Model-View-Controller (MVC) mimari deseni (architectural pattern) ilk kez 1979 yılında Trygve Reenskaug tarafından "Applications Programming in Smalltalk-80: How to use Model-View-Controller" makalesiyle tanımlanmıştır. Günümüze kadar bu desenin "Model View Presenter-MVP" gibi birçok türevi geliştirilmiştir.

Bu mimari deseni kullanmak, iş süreci ile ilgili olan katmanın gösterim katmanından bağımsız tasarlanmasını sağlar. Bunun yanında sunum katmanını bağımsız olarak kontrol eden bir nesneye sahip oluruz. Bu nedenle bu desene uygun bağımsız yapılan katmanların bakımı kolaylaşır, hem de **yeniden kullanımı** (reusing) artırılmış olunur.

MVC desenini Borland firması Visual Component Library (VCL) ve Component Library for Cross Platform (CLX) bileşenleri altyapısında yıllarca kullanmıştır. Benzer şekilde Apple firması Cocoa için bu deseni kullanmıştır. Microsoft firması da Document/View mimarisi olarak yıllarca kullanmıştır. Günümüzde .NET Framework Web Uygulamaları bu kapsamda daha kolay ve yönetilebilir olarak geliştirilmektedir.

MVC Deseni Temel Yapısı

MVC deseni adından da anlaşıldığı üzere üç nesneden veya katmandan oluşur. Bunlar aşağıda detaylı olarak açıklanmıştır;

1. **Model** : Sadece işlevsel davranışları yerine getiren kodlar burada toplanır. Kullanıcıya gösterilen ara yüzde kullanıcının gördüğü ve isteklerini ilettiği kısım dışında kalan konular bu katmanda gerçekleştirilir. Bazen kullanıcı tarafından modelin o andaki **durumu** (state) hakkında bilgi almasına izin verilir. **Gösterimler** (views) bilgiyi modelden aldığından, önceden modele **kayıt** (subscribe) olmak zorundadır. Bunun yanında model üzerinde bilgi değişikliği olduğunda bütün gösterimler bundan **haberdar edilir** (notify).
2. **Gösterimler** (view): MVC deseninde birden fazla gösterim olmasına izin verilir. Gösterim, modele ilişkin bir kullanıcı ara yüzüdür (**Graphical User Interface-GUI**). Yani modelden kullanıcıya gösterilecek değerleri alır ve kullanıcıya gösterir.
3. **Denetçiler** (controller): Denetçi, kullanıcı tarafından etkileşilen gösterimden gelen isteklere göre modeli günceller. Denetçi modeli güncellemek için, modelin değişik yöntemlerini çağırır ve modelin kendi **durumunu** (state) değiştirmesini sağlar. Buna bağlı olarak da model, tüm gösterimleri bundan haberdar eder.

Yukarıda kabaca çalışma prensibi verilen desende, model değiştiğinde gösterimlere haber verilmesi işlemi, Gözlemci Deseni ile yapılır. Bu durumun da ilave edildiği MVC deseni örneği aşağıda verilmiştir.

Gözlemci ile MVC Deseni İçin Örnek Proje

Bu deseni bir bardak ve bu bardağı dolduran garson örneğini ele alarak inceleyelim. MVC deseni için bardağı dolduracak ya da boşaltacak olan garson, denetçi; Doldurulacak ve boşaltılacak olan bardak ise modeldir. Bunun için Visual Studio üzerinde ile bir Windows uygulaması örnek olarak oluşturulacaktır. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

MVCWindowsUygulaması	→ Çözüm (Solution)
└ WinFormsApp1	→ Proje (Project)
└ IController.cs	→ Interface (Controller)
└ IModel.cs	→ Interface (Model)
└ IView.cs	→ Interface (IView)
└ Controllers.cs	→ Class (Controller)
└ Models.cs	→ Class (Model)
└ ProgressBarView.cs	→ UserControl (View)
└ PieView.cs	→ UserControl (View)
└ Form1.cs	→ Form (Windows Form)
└ Program.cs	→ Ana (Main) Program

Projeyi oluşturmak için;

1. Visual Studio üzerinde *File→New→Project...* seçilerek **MVCWindowsUygulaması** adlı çözüm içinde **WinFormsApp1** adlı ilk projemiz *Windows Form App* olarak oluşturulur.
2. Oluşan projede **Form1** adlı bir form ve **Program.cs** dosyasını otomatik olarak oluşur.

Projede MVC Ara Yüzlerinin Oluşturulması

3. Solution Explorer üzerinde projeye sağ tıklanarak *Add→New→New Item...* seçilerek *Interface* seçilir ve dosya adı **IController.cs** oluşturulur ve aşağıdaki şekilde değiştirilir;

```
public interface IController
{
    //Denetçinin (garsonun) Davranışları
    void AlınanDoldurmaİsteği(int pArtırımOranı);
    void AlınanBoşaltmaİsteği(int pEksilmeOranı);
}
```

4. Solution Explorer üzerinde projeye sağ tıklanarak *Add→New→New Item...* seçilerek *Interface* seçilir ve dosya adı **IModel.cs** oluşturulur ve aşağıdaki şekilde değiştirilir;

```
/*
Doluluk oranı, en az, en çok ne kadar doldurulacağı ve adı belirlenecek bir bardağın yani
modelin, denetçiden (garson) gelen istekleri yerine getirir.
*/
public interface IModel
{
    //Özellikler
    string Adı { get; set; }
    int DolulukOranı { get; }
    int EnAz { get; }
    int EnÇok { get; }
    //Davranışlar
    void Doldur(int pArtırımOranı);
    void Boşalt(int pEksilmeOranı);
}
```

5. Solution Explorer üzerinde projeye sağ tıklanarak *Add→New→New Item...* seçilerek *Interface* seçilir ve dosya adı **IView.cs** oluşturulur ve aşağıdaki şekilde değiştirilir;

```
/*
Bardağın doluluk oranını gösteren bir gösterime ilişkin ara yüzü aşağıdaki gibi
tanımlayabiliriz. Burada tanımlanan ara yüz, denetçinin görevlerinin bir kısmının ortaya
çıkartır. Yani dolu olan bir bardak için doldurma isteğine ve boş olan bir bardak için
boşaltma isteğine gösterim izin vermemelidir.
*/
public interface IView
{
    //View Davranışları
    void DoldurmaAktif();
    void DoldurmaPasif();
    void BoşaltmaAktif();
    void BoşaltmaPasif();
}
```

6. **IController** ara yüzümüze hangi model ile çalışacağını söyleyen değişiklikler de yapılır.

```
public interface IController
{
    //Denetçinin (garsonun) Davranışları
    void AlınanDoldurmaİsteği(int pArtırımOranı);
    void AlınanBoşaltmaİsteği(int pEksilmeOranı);
    IModel Model { set; get; }
}
```

Projeye Gözlemci Deseninin Uygulanması

7. Bundan sonraki aşama biraz daha ustalık istemektedir. Çünkü modeldeki değişikliklerden gösterimin haberi olmasını istiyoruz. İşte bunun için *Gözlemci* desenini bu ara yüze uyguluyoruz. Bunun için ilk önce **IModel** ara yüzü aşağıdaki şekilde değiştirilir;

```
/*
Doluluk oranı, en az, en çok ne kadar doldurulacağı ve adı belirlenecek bir bardağın yani
modelin, denetçiden (garson) gelen istekleri yerine getirir.
*/
using System.Collections; //ArrayList İçin
public interface IModel
{
    //Özellikler
    string Adı { get; set; }
    int DolulukOranı { get; }
    int EnAz { get; }
    int EnÇok { get; }
    //Davranışlar
    void Doldur(int pArttırımOranı);
    void Boşalt(int pEksilmeOranı);
    //Gözlemci Desenine İlişkin Davranışlar
    ArrayList Observers { get; }
    void AddObserver(IView pView);
    void RemoveObserver(IView pView);
    void NotifyObservers();
}
```

8. Devamında **IView** ara yüzü de aşağıdaki şekilde değiştirilir;

```
/*
Bardağın doluluk oranını gösteren bir gösterime ilişkin ara yüzü aşağıdaki gibi
tanımlayabiliriz. Burada tanımlanan ara yüz, denetçinin görevlerinin bir kısmının ortaya
çıkartır. Yani dolu olan bir bardak için doldurma isteğine ve boş olan bir bardak için
boşaltma isteğine gösterim izin vermemelidir.
*/
public interface IView
{
    //View Davranışları
    void DoldurmaAktif();
    void DoldurmaPasif();
    void BoşaltmaAktif();
    void BoşaltmaPasif();
    //Gözlemci Deseni Davranışları:
    //Modelden gelen bilgilere göre View güncellenir
    void Update(IModel pModel);
}
```

9. Son olarak gösterim ara yüzünden alınacak kullanıcı isteklerini denetçiye iletmek için **Iview** ara yüzü aşağıdaki gibi değiştirilir;

```
/*
Bardağın doluluk oranını gösteren bir gösterime ilişkin ara yüzü aşağıdaki gibi
tanımlayabiliriz. Burada tanımlanan ara yüz, denetçinin görevlerinin bir kısmının ortaya
çıkartır. Yani dolu olan bir bardak için doldurma isteğine ve boş olan bir bardak için
boşaltma isteğine gösterim izin vermemelidir.
*/
public interface IView
{
    //View Davranışları
    void DoldurmaAktif();
    void DoldurmaPasif();
    void BoşaltmaAktif();
    void BoşaltmaPasif();
}
```

```
//View üzerinden kullanıcı isteklerini Controller'e
//iletme için özellik tanımlama
IController Control { set; }
//Gözlemci Deseni Davranışları:
//Modelden gelen bilgilere göre View güncellenir
void Update(IModel pModel);
}
```

MVC İskeletine Uygun Projenin Tamamlanması

Artık verilen örneği gerçekleştirmek için iskelet ara yüzlerimiz hazır. Bu ara yüzlere bağlı kalarak sırasıyla soyut ve somut bir model (bardak), denetçi (hızlı veya çalışan garson), ve gösterimler (**ProgressBarView**, **PieView**) oluşturabiliriz.

10. Solution Explorer üzerinde projemize sağ tıklanarak *Add→New→New Item...* seçilerek **Models.cs** adında *Class* seçilir ve içeriği aşağıdaki şekilde değiştirilir;

```
using System.Collections; //ArrayList İçin
abstract class SoyutKap: IModel
{
    private ArrayList views = new ArrayList();
    private string adi;
    private int dolulukOranı;
    private int enAz;
    private int enÇok;
    public SoyutKap(string pAdi, int pEnAz, int pEnÇok,
        int pDolulukOranı)
    {
        this.adi = pAdi;
        this.enAz = pEnAz;
        this.enÇok = pEnÇok;
        this.dolulukOranı = pDolulukOranı;
    }
    public string Adı
    {
        get { return adi; }
        set { adi = value; }
    }
    public int DolulukOranı
    {
        get { return dolulukOranı; }
    }
    public int EnAz
    {
        get { return enAz; }
    }
    public int EnÇok
    {
        get { return enÇok; }
    }
    public void Doldur(int pArtırımOranı)
    {
        this.dolulukOranı += pArtırımOranı;
        if (dolulukOranı >= this.enÇok) dolulukOranı = enÇok;

        this.NotifyObservers();
    }
    public void Boşalt(int pEksilmeOranı)
    {
        this.dolulukOranı -= pEksilmeOranı;
        if (dolulukOranı <= this.enAz) dolulukOranı = enAz;

        this.NotifyObservers();
    }
}
```

```

    }
    public ArrayList Observers
    {
        get { return views; }
    }
    public void AddObserver(IView pView)
    {
        views.Add(pView);
    }
    public void RemoveObserver(IView pView)
    {
        views.Remove(pView);
    }
    public void NotifyObservers()
    {
        foreach (IView view in views)
        {
            view.Update(this);
        }
    }
}
class somutBardak : SoyutKap
{
    public somutBardak(string pAdi) : base(pAdi, 0, 100, 0) { }
}

```

11. Solution Explorer üzerinde projemize sağ tıklanarak *Add→New→New Item...* seçilerek **Controllers.cs** adında *Class* seçilir içeriği aşağıdaki şekilde değiştirilir;

```

class HızlıGarson : IController
{
    private IModel model;
    public IModel Model
    {
        set { model = value; }
        get { return model; }
    }
    public void AlınanDoldurmaİsteği(int pArttırımOranı)
    {
        if (model != null)
        {
            model.Doldur(pArttırımOranı * 10);
            DurumKontrol();
        }
    }
    public void AlınanBoşaltmaİsteği(int pEksilmeOranı)
    {
        if (model != null)
        {
            model.Boşalt(pEksilmeOranı * 10);
            DurumKontrol();
        }
    }
    public void DurumKontrol()
    {
        foreach (IView view in model.Observers)
        {
            if (model.DolulukOranı >= model.EnÇok)
                view.DoldurmaPasif();
            else
                view.DoldurmaAktif();

            if (model.DolulukOranı <= model.EnAz)

```

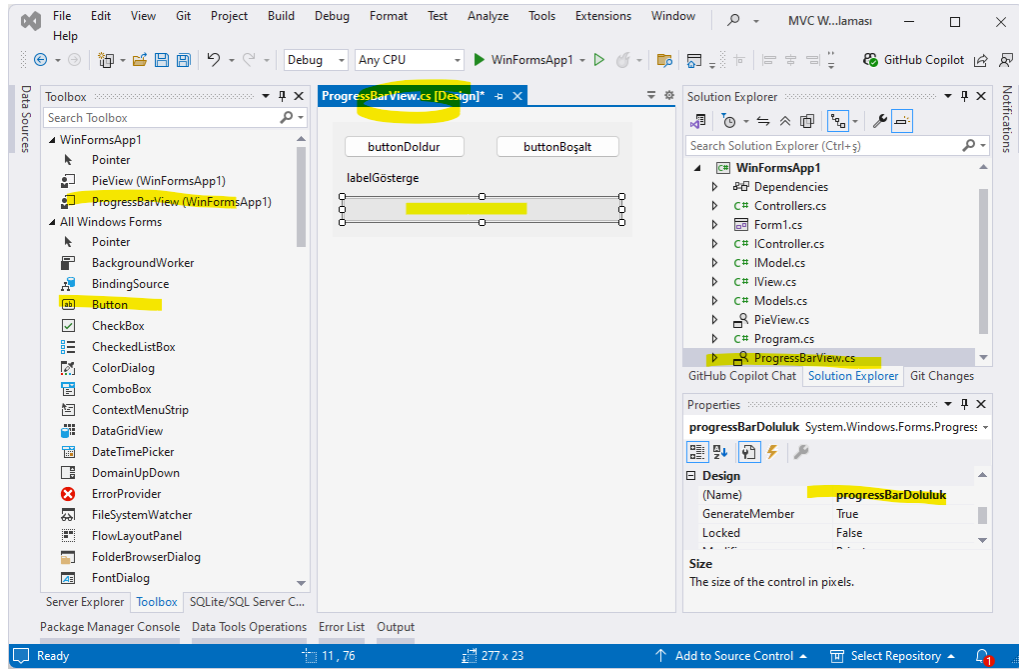
```

        view.BoşaltmaPasif();
    else
        view.BoşaltmaAktif();
    }
}
}
class YavaşGarson : IController
{
    private IModel model;
    public IModel Model
    {
        set { model = value; }
        get { return model; }
    }
    public void AlınanDoldurmaİsteği(int pArtırımOranı)
    {
        if (model != null)
        {
            model.Doldur(pArtırımOranı);
            DurumKontrol();
        }
    }
    public void AlınanBoşaltmaİsteği(int pEksilmeOranı)
    {
        if (model != null)
        {
            model.Boşalt(pEksilmeOranı);
            DurumKontrol();
        }
    }
    public void DurumKontrol()
    {
        foreach (IView view in model.Observers)
        {
            if (model.DolulukOranı >= model.EnÇok)
                view.DoldurmaPasif();
            else
                view.DoldurmaAktif();

            if (model.DolulukOranı <= model.EnAz)
                view.BoşaltmaPasif();
            else
                view.BoşaltmaAktif();
        }
    }
}
}

```

12. Artık gösterimler için örnek olarak iki farklı kullanıcı kontrolü (user control) tasarlanacaktır. Bunlardan biri **ProgressBarView** kontrolüdür.
13. Projemize kullanıcı kontrolü eklemek için Solution Explorer üzerinde projemize sağ tıklanarak *Add→New→New Item...* ile **ProgressBarView.cs** adında *User Control (Windows Forms)* seçilir.
14. Daha sonra *View→ToolBox* tıklanır ve açılan penceren kullanıcı kontrolüne *Toolbox* üzerinden sürükleyip bırak ile iki adet **Button** ve bir adet **ProgressBar** eklenir;



Şekil 58. ProgressBarView Kullanıcı Kontrolü

15. Eklenen kontrollerin **Name** özellikleri **buttonDoldur**, **buttonBoşalt**, **labelGösterge** ve **progressBarDoluluk** olarak değiştirilir ve butonların **onClick()** olayına çift tıklanır. Açılan **ProgressBarView.cs** dosyasında aşağıdaki değişiklikler yapılır;

```
namespace WinFormsApp1
{
    public partial class ProgressBarView : UserControl, IView
    {
        public ProgressBarView()
        {
            InitializeComponent();
        }

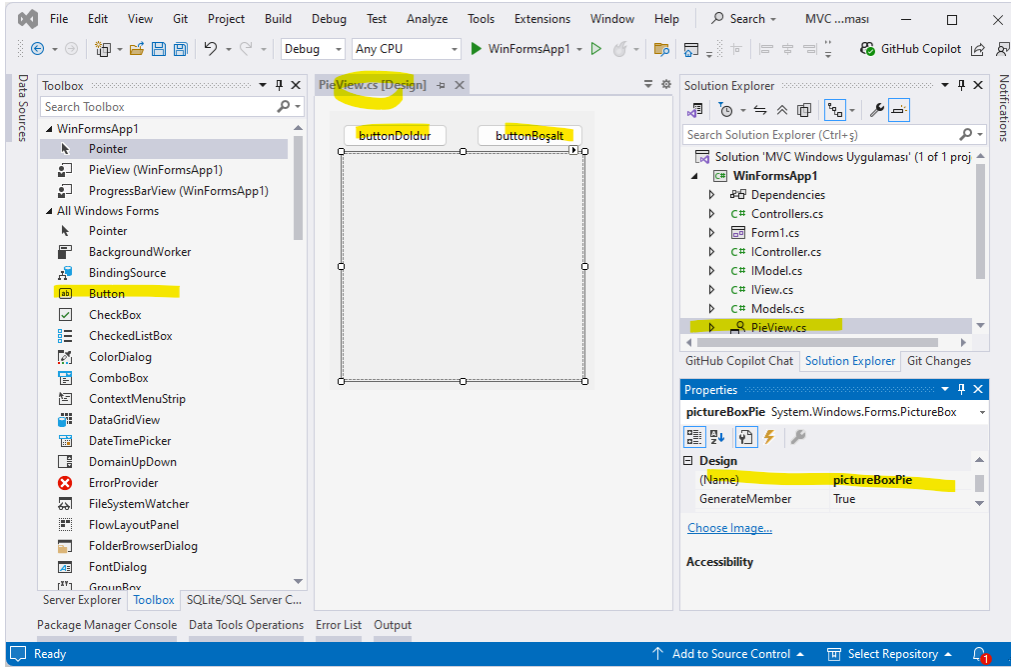
        private IController control;
        [DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]
        public IController Control
        {
            set { control = value; }
        }
        public void DoldurmaAktif()
        {
            this.buttonDoldur.Enabled = true;
        }
        public void DoldurmaPasif()
        {
            this.buttonDoldur.Enabled = false;
        }
        public void BoşaltmaAktif()
        {
            this.buttonBoşalt.Enabled = true;
        }
        public void BoşaltmaPasif()
        {
            this.buttonBoşalt.Enabled = false;
        }
        public void Update(IModel pModel)
        {
            this.labelGösterge.Text = string.Format(
                "{0} Doluluğu (%):{1}", pModel.Adı,
```

```

        pModel.DolulukOranı);
        this.progressBarDoluluk.Value = pModel.DolulukOranı
            * 100 / pModel.EnÇok;
    }
    private void buttonBoşalt_Click(object sender, EventArgs e)
    {
        control.AlınanBoşaltmaİsteği(1);
    }
    private void buttonDoldur_Click(object sender, EventArgs e)
    {
        control.AlınanDoldurmaİsteği(1);
    }
}
}

```

16. İkinci kullanıcı kontrolü ise **PieView** kontrolüdür. Bunun için mevcut projemize **PieView** adında yeni bir kullanıcı kontrolü eklenir.
17. Kullanıcı kontrolüne *Toolbox* üzerinden sürükleyip bırak ile iki adet **Button**, ve bir adet **PictureBox** eklenir.



Şekil 59. PictureBox Kullanıcı Kontrolü

18. Eklenen kontrollerin **Name** özellikleri **buttonDoldur**, **buttonBoşalt** ve **pictureBoxPie** olarak değiştirilir ve butonların **onClick()** olayına çift tıklanır. Açılan **PieView.cs** dosyasında aşağıdaki değişiklik yapılır;

```

namespace WinFormsApp1
{
    public partial class PieView : UserControl, IView
    {
        public PieView()
        {
            InitializeComponent();
        }
        private IController control;
        [DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]
        public IController Control
        {
            set { control = value; }
        }
        public void DoldurmaAktif()
    }
}

```



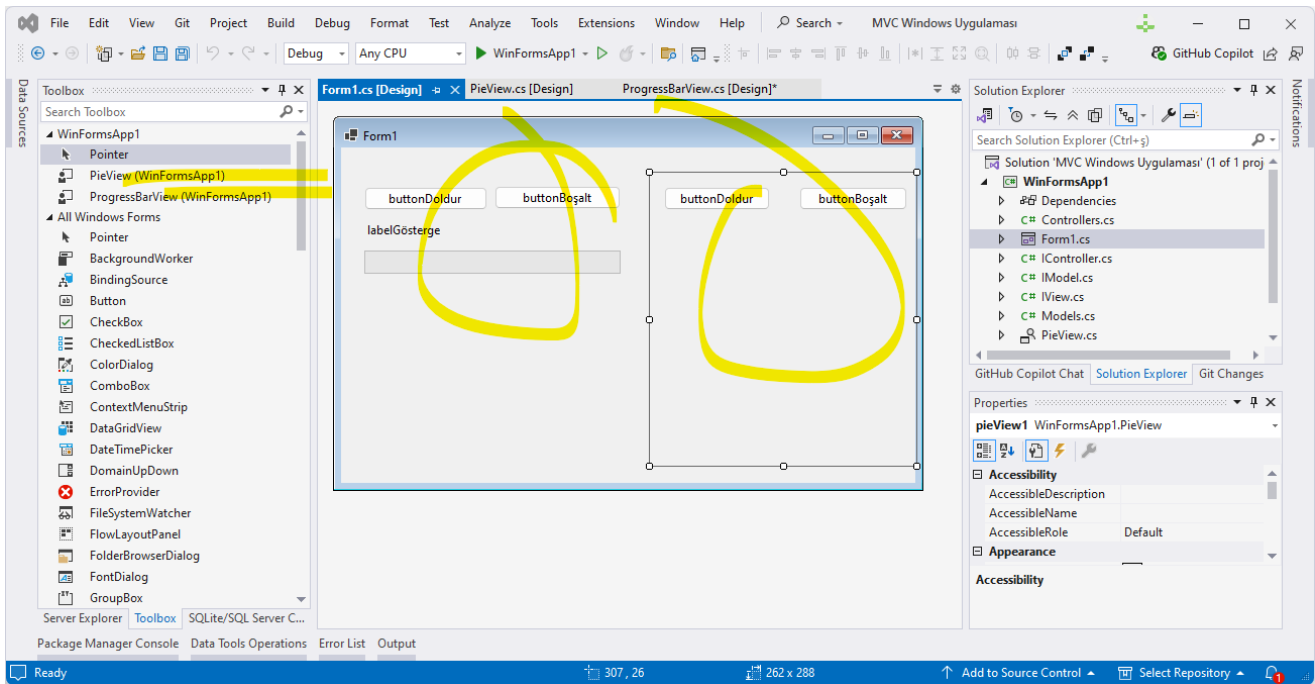
```

    {
        this.buttonDoldur.Enabled = true;
    }
    public void DoldurmaPasif()
    {
        this.buttonDoldur.Enabled = false;
    }
    public void BoşaltmaAktif()
    {
        this.buttonBoşalt.Enabled = true;
    }
    public void BoşaltmaPasif()
    {
        this.buttonBoşalt.Enabled = false;
    }
    public void Update(IModel pModel)
    {
        SolidBrush whiteBrush = new SolidBrush(Color.White);
        SolidBrush redBrush = new SolidBrush(Color.Red);
        Graphics graphics = this.pictureBoxPie.CreateGraphics();
        graphics.FillRectangle(whiteBrush, 0, 0,
            this.pictureBoxPie.Width,
            this.pictureBoxPie.Height);
        graphics.FillPie(redBrush, 0, 0,
            this.pictureBoxPie.Width,
            this.pictureBoxPie.Height, 0.0F,
            pModel.DolulukOranı * 360.0F / pModel.EnÇok);
        Font font = new Font(FontFamily.GenericSerif, 12);
        Point point = new Point(this.pictureBoxPie.Width / 2,
            this.pictureBoxPie.Height / 2);
        SolidBrush blackBrush = new SolidBrush(Color.Black);
        graphics.DrawString(string.Format("%{0}",
            pModel.DolulukOranı), font, blackBrush, point);
    }
    private void buttonBoşalt_Click(object sender, EventArgs e)
    {
        control.AlınanBoşaltmaİsteği(1);
    }
    private void buttonDoldur_Click(object sender, EventArgs e)
    {
        control.AlınanDoldurmaİstegi(1);
    }
}
}

```

19. Kullanıcı Kontrollerimizi tanımladıktan sonra uygulamanın asıl form olan **Form1.cs** formuna dönüp bu kontrolleri ekleyebiliriz.

20. Bu forma kullanıcı kontrollerimizi *Toolbox* üzerinden sürükleyip bırak ile ekleyebiliriz.



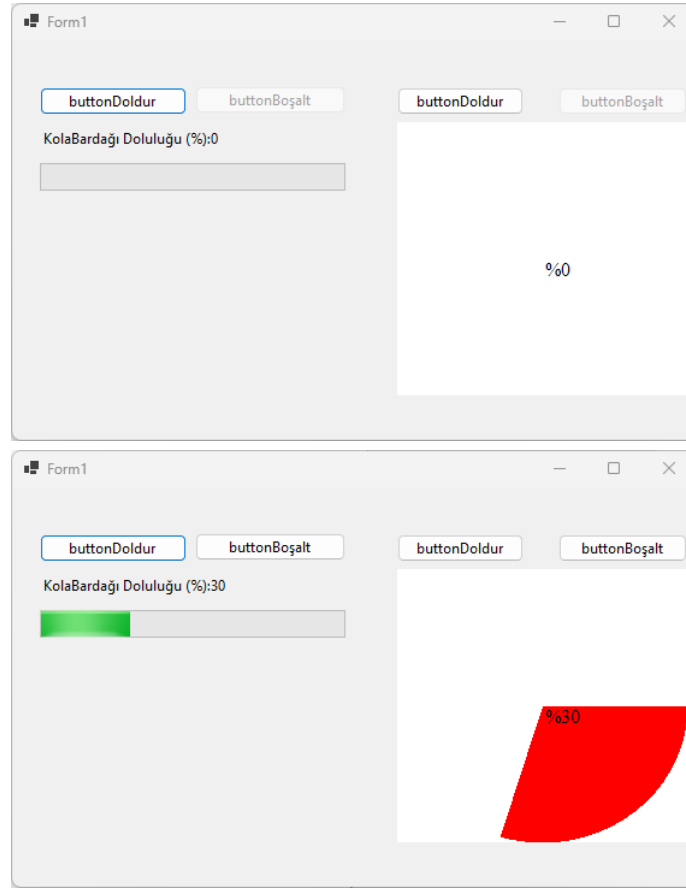
Şekil 60. WinFormApp1 Uygulamasında Form1

21. Formun kod kısmına geçerek **Form1.cs** dosyasında aşağıdaki değişiklikleri yaparak programı son haline getiririz.

```
namespace WinFormsApp1
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
            MVCBaşlat();
        }
        public void MVCBaşlat()
        {
            //Controller
            IController garson = new HızlıGarson();
            //Model
            IModel bardak = new somutBardak("KolaBardağı");
            //Denetçinin Modeli
            garson.Model = bardak;
            //Her bir Gösterime Denetçi Ataması
            pieView1.Control = garson;
            progressBarView1.Control = garson;
            //Modele Gösterimlerin Kayıt Olması
            bardak.AddObserver(pieView1);
            bardak.AddObserver(progressBarView1);
        }
    }
}
```

Program çalıştırıldığında, MVC deseni ile **bağlaşımı az olan (loosely coupled)** denetleme ara yüzlerinin yazılımlara kolay eklendiğini görebiliriz. Ayrıca programda yapılacak değişiklik isteklerine MVC deseni daha hızlı cevap verecektir. Bunun yanı sıra hazırlanan ara yüzlerle birlikte, soyut sınıfları da başka projelere taşıyıp **yeniden kullanmak (reusing)** oldukça kolay olacaktır. Böylece ileride olabilecek değişikliklere daha hızlı ayak uydurabilen daha esnek bir projeye sahip olmuş oluruz. Sonuç olarak MVC deseni, kullanıcı ara yüzü işlemleri ile iş mantığının birbirinden ayrılmasını sağlar. Ayrıca Model,

Gösterim ve Denetçi bileşenleri ayrı katmanlar halinde *Katmanlı Mimari* başlığı altında anlatıldığı şekilde tasarlanabilir.



Şekil 61. MVC Uygulanmış Windows Form Uygulamamız.

ASP.NET MVC ve Jakarta EE gibi günümüzde en çok kullanılan teknolojiler MVC ara yüzlerini web uygulamasına entegre edilmiş olarak sunmaktadır. ASP.NET MVC altyapısında gösterim bileşeni ASP.NET web sayfası olacak şekilde hazırlanır. Jakarta EE altyapısında ise sunum bileşeni JSP olacak şekilde hazırlanır. Model ve Denetçi bileşenleri ise genellikle nesneye dayalı sınıflar olarak tasarlanır.